



Algorithmes proximaux déroulés pour l'estimation du nombre de reproduction de la pandémie de COVID-19

RAPPORT DE STAGE INGÉNIEUR
31/03/2025 - 29/08/2025
LS2N, ECOLE CENTRALE DE NANTES



Etudiant :
BENJAMIN PINEAU
benjamin.pineau@eleves.ec-nantes.fr

Encadrants :
BARBARA PASCAL
Chargée de recherche CNRS
barbara.pascal@cnrs.fr
SÉBASTIEN BOURGUIGNON
Professeur à Centrale Nantes
sebastien.bourguignon@ec-nantes.fr

Remerciements

Je souhaite tout d'abord remercier le laboratoire LS2N ainsi que l'équipe SIMS, qui m'ont accueilli chaleureusement durant ce stage. J'adresse mes remerciements particuliers à Barbara Pascal et Sébastien Bourguignon pour leur accompagnement constant, leurs conseils avisés et le partage de leur expertise, qui ont grandement enrichi mon travail et ma compréhension du sujet.

Je remercie également l'ANR OPTIMOCSI¹ pour m'avoir offert l'opportunité d'effectuer ce stage dans un cadre stimulant et propice à la recherche.

Enfin, je souhaite remercier l'ANR GEODSIC² pour l'organisation de la conférence *Geometry In Data*, qui m'a permis d'élargir mes compétences scientifiques et de découvrir le déroulement concret d'une conférence scientifique, une expérience précieuse pour la suite de mon parcours.

1. optimocsi.cnrs.fr

2. projetanrgeodsic/accueil

Table des matières

1	Introduction	1
2	Contexte et état-de-l'art	4
2.1	Surveillance d'une épidémie	4
2.1.1	Modèle épidémiologique	4
2.1.2	Estimateurs du nombre de reproduction	5
2.2	Résolution d'un problème d'optimisation convexe non lisse	7
2.3	Apprentissage statistique et réseaux de neurones déroulés	9
3	Estimation du nombre de reproduction via un algorithme proximal déroulé	13
3.1	Déroulement de l'algorithme de Chambolle-Pock	13
3.2	Evaluation des capacités d'approximation du réseau déroulé	17
3.2.1	Stratégie d'apprentissage s'appuyant sur un oracle	18
3.2.2	Ajustement du paramètre de régularisation	19
3.3	Capacités d'approximation de plusieurs jeux de poids	21
3.3.1	Comparaison de l'apprentissage de plusieurs jeux de poids	21
3.3.2	Stratégie d'adaptation du pas d'apprentissage	24
4	Capacités de généralisation du réseau déroulé	26
4.1	Construction de la base de données	26
4.2	Ajustement de l'architecture	27
4.2.1	Profondeur du réseau déroulé	27
4.2.2	Généralisation de l'ajustement du pas d'apprentissage	27
4.3	Apprentissage sur une base de données	27
4.3.1	Ajustement à plusieurs séries temporelles	27
4.3.2	Etude de la capacité de généralisation	29
5	Conclusions et perspectives	30
	Références	31
A	Notations	33
B	Descente de gradient et Adam	34
B.1	Principe général	34
B.2	Algorithme de base	34
B.3	Algorithme Adam	34
C	Concepts et outils d'optimisation convexe non lisse	36
D	Stratégie d'adaptation du pas d'apprentissage	39
E	Illustration des données	39

1 Introduction

La pandémie de COVID-19 a souligné la nécessité de disposer d'outils précis, robustes aux erreurs et retards dans le report du nombre de cas ; et capables d'évaluer en temps réel l'intensité d'une épidémie. En pratique, lors d'une épidémie les autorités sanitaires collectent quotidiennement le nombre de nouvelles infections. On peut par exemple retrouver en Figure 1 le nombre quotidien d'infections, noté Z_t au jour t en France sur l'année 2022. En France, ces données ont été reportées par Santé Publique France. A plus large échelle, l'Université Johns Hopkins a collecté et mis à disposition les nombres de cas de COVID-19 reportés quotidiennement par les agences de santé publique dans plus de 200 pays durant près de 3 ans de pandémie.³ (JHU).

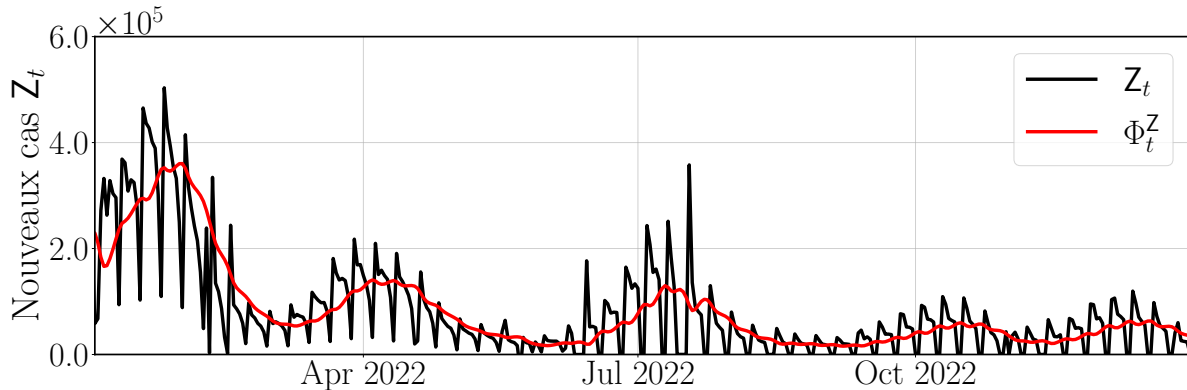


FIGURE 1 – *En noir* : décompte journalier des nouveaux cas de COVID-19 en France, collectés par l'Université Johns Hopkins. *En rouge* : infectiosité globale.

Cependant, cette donnée brute demeure insuffisante pour appréhender pleinement la dynamique d'évolution de l'épidémie ainsi que son intensité : en effet, au jour t le nombre de nouvelles infections est une conséquence de la propagation du virus dans la population durant les semaines précédentes. Afin de quantifier précisément l'intensité de la propagation, les épidémiologistes recourent à des indicateurs plus avancés. Au début d'une épidémie, les autorités sanitaires s'intéressent au nombre de reproduction de base R_0 qui indique le nombre moyen de nouveaux cas qu'une seule personne infectée va générer en moyenne dans une population sans aucune immunité et sans mesure de contrôle. Ainsi si R_0 est inférieur à un, une personne malade contaminera strictement moins d'une personne en moyenne, et finira par disparaître. En revanche si ce dernier est supérieur à un, une personne malade contaminera en moyenne plus d'une personne, provoquant une croissance exponentielle du nombre de cas : c'est ce que l'on appelle une vague épidémique. Afin de suivre l'intensité d'une épidémie en temps réel, une généralisation du nombre de reproduction de base a été proposée : le nombre de reproduction effectif noté R_t [1]. Pour un individu infecté à la date t , il désigne le nombre moyen de personne que ce dernier va infecter durant sa période de contagiosité dans une population composée à la fois de personnes susceptibles et de personnes immunisées [2]. Ce taux de reproduction peut être calculé à chaque instant t pour estimer la dynamique de l'épidémie à partir du nombre de cas au jour t ainsi que des comptes passés durant la période de contagiosité. L'objectif des autorités sanitaires est donc de maintenir R_t en dessous de un afin de maîtriser l'intensité des vagues épidémiques.

3. coronavirus.jhu.edu/

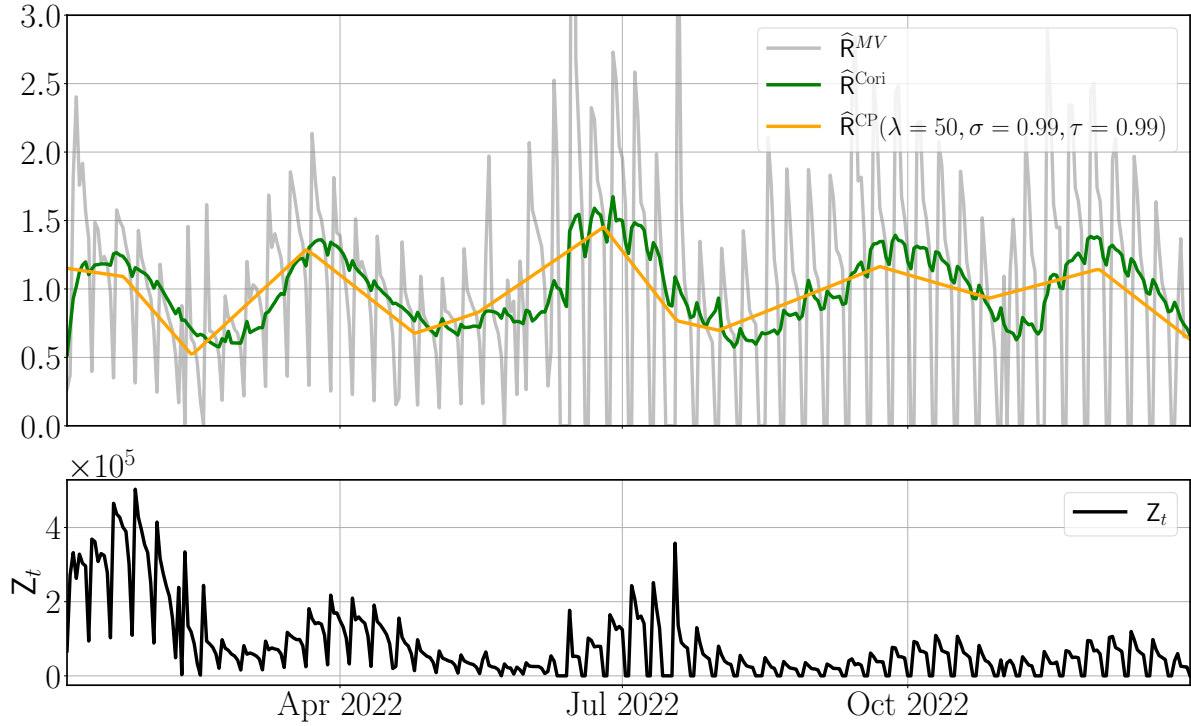


FIGURE 2 – Première ligne : estimateurs du nombre de reproduction R_t . *En gris* : estimateur du maximum de vraisemblance (1). *En vert* : moyenne a posteriori pour un modèle bayésien avec vraisemblance de Poisson et une loi a priori Gamma et régularisation temporelle. *En orange* : estimateur du maximum de vraisemblance pénalisée. Deuxième ligne : décompte journalier des nouveaux cas Z_t .

La manière exacte d'estimer R_t à partir des comptes dépend du modèle épidémiologique utilisé. Dans ce travail, nous considérerons le modèle proposé par A. Cori et ses collaborateurs dans [2], [3]⁴, qui sera décrit en détail en Section 2.1.1. Un estimateur naïf de R_t obtenu à partir du maximum de vraisemblance de ce modèle est représenté en gris en Figure 2. Cet estimateur n'est pas adapté au suivi en temps réel d'une pandémie telle que celle de COVID-19 pour plusieurs raisons. D'une part, l'estimateur du maximum de vraisemblance ne rend pas compte de la véritable évolution du nombre de reproduction : il est difficilement concevable que R_t varie aussi fortement en quelques jours. Les performances limitées de cet estimateur sont dues au fait que l'on a estimé T inconnues à partir de T observations. De plus, ces observations sont de mauvaise qualité : Z_t est affecté par un bruit administratif, se traduisant notamment par des valeurs très faibles, voire nulles, les week-ends en raison de la fermeture de certains laboratoires d'analyse et de l'interruption des canaux de collecte les jours fériés. D'autre part, l'objectif des autorités est de maintenir ce nombre de reproduction en dessous de un. Pour que celles-ci puissent prendre des décisions, il faut leur fournir un estimateur facile à lire et à comprendre permettant non seulement de mesurer l'intensité de l'épidémie à un instant t mais également de lire les grandes tendances d'évolution de cet indicateur, ce qui n'est pas le cas ici. Il est donc nécessaire de recourir à des estimateurs palliant la mauvaise qualité des données afin d'obtenir un indicateur ayant un comportement plus réaliste du point de vue de l'épidémiologie, à savoir variant suffisamment continûment dans le temps. A. Cori et ses collègues ont proposé un estimateur bayésien régularisé en temps [2], tracé en vert sur

4. github.com/jstockwin/EpiEstimApp

la Figure 2. Cependant cet estimateur n’arrive toujours pas à s’adapter à la situation inédite de la COVID-19 avec beaucoup de bruit administratif, et c’est alors que des estimateurs variationnels favorisant le comportement linéaire par morceaux ont été introduits [4], [5]. La linéarité par morceaux favorisera non seulement la régularité en temps, mais permettra également de dégager des grandes tendances, facilitant la lecture de l’estimée. Plus précisément, il s’agit de trouver un estimateur dont les dérivées secondes sont parcimonieuses : en minimisant la norme ℓ_1 de la dérivée seconde de R_t , l’estimateur tendra à être un estimateur linéaire par morceaux. Ces méthodes variationnelles combinent une attache aux données utilisant la vraisemblance du modèle proposé par [2] pour mesurer l’adéquation des comptes reportés au modèle épidémiologique et une pénalisation en norme ℓ_1 pour favoriser la parcimonie des estimées. L’estimateur résultant est alors défini comme la solution d’un problème d’optimisation convexe non lisse, qui peut être résolu en faisant appel à la théorie des opérateurs proximaux dont les principaux outils sont décrits en Annexe C. Le schéma primal-dual de Chambolle-Pock a été utilisé dans [4] et [5], donnant l’estimée en orange sur la Figure 2 qui est bien plus réaliste que les deux précédents. Il y a cependant deux obstacles majeurs à l’utilisation à très grande échelle de cet estimateur pour surveiller l’épidémie simultanément sur un grand nombre de territoires. Premièrement les estimateurs variationnels nécessitent le réglage d’hyperparamètres contrôlant de manière cruciale le niveau de régularisation temporelle et nous ne pouvons pas demander à un expert de faire ce réglage pays par pays. Ensuite la convergence des algorithmes proximaux est lente : nous avons donc une limitation algorithmique. Une idée nouvelle est de déplier l’algorithme d’optimisation sous forme d’un réseau de neurones comme proposé dans [6]. Cela permet d’apprendre de manière supervisée les paramètres de régularisation mais également les éléments du modèle, tels que le type de régularité temporelle optimal au delà de la linéarité par morceaux. Nous bénéficions par ailleurs de l’implémentation efficace et rapide des réseaux de neurones en GPU.

La contribution principale de ce travail est le dépliement de l’algorithme primal-dual de Chambolle-Pock utilisé pour l’estimation du nombre de reproduction d’une épidémie viral [4], [5]. L’originalité majeure du réseau obtenu est de rendre compte de manière exacte du caractère poissonnien du modèle épidémiologique là où la plupart des réseaux déroulés ne considèrent que le cas gaussien. Une seconde contribution est l’élaboration d’un protocole rigoureux de comparaison des capacités d’approximation du réseau en fonction de sa paramétrisation. Enfin, les capacités de généralisation du réseau déroulé sur des données réelles de comptes d’infection à la COVID-19 dans plusieurs pays sont étudiées. Ce rapport porte sur le dépliement de l’algorithme de Chambolle-Pock de [4] avec une attache aux données particulière à notre problème, l’élaboration d’un cadre expérimental pour déterminer le niveau d’adaptation de l’architecture et le nombre de paramètres à apprendre pour pouvoir produire des estimées suffisamment régularisées, mais également sur les capacités de généralisation de l’architecture.

En Section 2, le modèle épidémiologique sera décrit en détail puis les différents estimateurs qui s’appuient sur ce modèle seront présentés. Nous fixerons également le cadre de l’apprentissage statistique et présenterons le principe de dépliement d’algorithme proximal sous forme de réseau de neurones à propagation avant. Dans la Section 3 nous réaliserons le dépliement d’un algorithme d’optimisation permettant d’obtenir l’un des estimateurs présenté en Section 2 dont l’architecture obtenue sera décrite en détail, et nous explorerons les capacités d’approximation du réseau. Nous étudierons enfin en Section 3.3 l’apprentissage d’estimateurs à partir de base de données. Nous terminerons ce rapport par une conclusion générale ainsi que par une ouverture sur les perspectives envisagées à la suite de ces travaux.

2 Contexte et état-de-l'art

2.1 Surveillance d'une épidémie

2.1.1 Modèle épidémiologique

Le décompte journalier des nouveaux cas de COVID-19, illustré à la Figure 1, n'est pas suffisant pour quantifier précisément l'intensité de l'épidémie à l'instant t , d'où la nécessité de recourir à des indicateurs épidémiologiques plus développés. Un indicateur beaucoup utilisé pour le suivi des épidémies virales est le nombre de reproduction effectif au temps t , noté R_t . Au jour t , le coefficient de reproduction effectif est défini comme le nombre moyen de cas secondaires générés par un individu contagieux typique durant toute sa période d'infectiosité [2]. Il est défini comme le nombre moyen d'infections secondaires au jour t causées par un individu contagieux. Ainsi lorsque $R_t > 1$ (respectivement $R_t < 1$), le nombre de nouveau cas croît (respectivement décroît) de manière exponentielle. Lorsque $R_t = 1$, l'épidémie est stable.

Dans le modèle épidémiologique proposé par [2], le nombre de cas au jour t est une variable aléatoire Z_t dont la loi conditionnelle sachant les nombres de cas passés $Z_1, \dots, Z_{t-1}$ est modélisée par une variable aléatoire de Poisson d'intensité $R_t \Phi_t^Z$ avec Φ_t^Z l'infectiosité globale au jour t , définie comme une moyenne pondérée des précédents nombres de cas (tracée en rouge sur la Figure 2) :

$$\Phi_t^Z = \sum_{s=1}^{\tau_\Phi} Z_{t-s} \phi_s$$

où ϕ est la fonction d'intervalle de série qui rend compte du caractère aléatoire du délai entre infection primaire et infection secondaire. La fonction d'intervalle de série est communément modélisée par une loi Gamma, dont les paramètres dépendent de l'agent pathogène et de la population au sein de laquelle il se propage. Le suivi des chaînes de transmission par les autorités sanitaires en début de pandémie a permis aux épidémiologistes d'estimer pour la COVID-19 une moyenne de 6,6 jours et un écart type de 3,5 jours, conduisant à une loi Gamma dont le support est essentiellement compris entre 1 et $\tau_\Phi = 25$ jours [7], [8]. La fonction d'intervalle de série de la COVID-19 est tracée en trait plein bleu en Figure 3. En outre, sachant que le nombre de cas à la date t , conditionnellement aux comptes passés suit une loi de Poisson de paramètre $R_t \Phi_t^Z$, nous obtenons la densité conditionnelle suivante :

$$\mathbb{P}(Z_t | Z_1, \dots, Z_{t-1}) = \frac{(R_t \Phi_t^Z)^{Z_t} \exp(-R_t \Phi_t^Z)}{Z_t!}$$

il en découle que

$$\mathbb{E}[Z_t | Z_1, \dots, Z_{t-1}] = R_t \Phi_t^Z, \quad \Phi_t^Z = \sum_{s=1}^{\tau_\Phi} Z_{t-s} \phi_s.$$

Nous pouvons en déduire la loi du vecteur $\mathbf{Z} = (Z_1, \dots, Z_T)$:

$$\mathbb{P}(\mathbf{Z}) = \prod_{s=1}^T \frac{(R_s \Phi_s^Z)^{Z_s} \exp(-R_s \Phi_s^Z)}{Z_s!}$$

L'enjeu est alors de trouver un estimateur du vecteur $\mathbf{R} = (R_1, \dots, R_T)$ à partir du vecteur $\mathbf{Z} = (Z_1, \dots, Z_T)$. Nous sommes alors dans le cas où nous avons autant d'inconnues que d'observations.

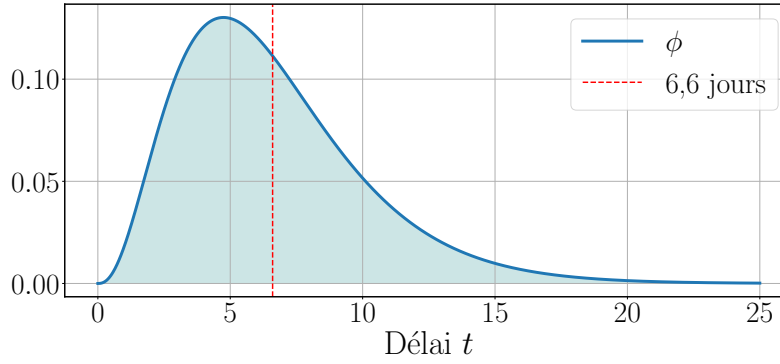


FIGURE 3 – Fonction d'intervalle de série de la COVID-19, de moyenne 6,6 jours (en rouge).

2.1.2 Estimateurs du nombre de reproduction

Maximum de vraisemblance

Nous sommes dans le cas où nous avons accès aux réalisations du vecteur $\mathbf{Z}$, et nous souhaitons trouver un estimateur du vecteur $\mathbf{R}$. Pour ce faire, l'estimateur le plus largement utilisé est l'estimateur du maximum de vraisemblance. Nous nous intéressons alors à la fonction de vraisemblance, et nous cherchons le maximum de cette vraisemblance qui donnera un estimateur du nombre de reproduction. Maximiser la vraisemblance est strictement équivalent à minimiser la log-vraisemblance, ce qui mène au calcul suivant :

$$\begin{aligned}
 \ln(\mathbb{P}(\mathbf{Z}_1, \dots, \mathbf{Z}_T | \mathbf{R}_1, \dots, \mathbf{R}_T)) &= \ln\left(\prod_{t=1}^T \mathbb{P}(\mathbf{Z}_t | \mathbf{Z}_1, \dots, \mathbf{Z}_{t-1}, \mathbf{R}_t)\right) \\
 &= \sum_{t=1}^T \ln(\mathbb{P}(\mathbf{Z}_t | \mathbf{Z}_1, \dots, \mathbf{Z}_{t-1}, \mathbf{R}_t)) \\
 &= \sum_{t=1}^T \mathbf{Z}_t \ln(\mathbf{R}_t \Phi_t^{\mathbf{Z}}) - \mathbf{R}_t \Phi_t^{\mathbf{Z}} - \ln(\mathbf{Z}_t!) \\
 &\underset{\mathbf{Z}_t \gg 1}{\simeq} \sum_{t=1}^T \mathbf{Z}_t \ln(\mathbf{R}_t \Phi_t^{\mathbf{Z}}) - \mathbf{R}_t \Phi_t^{\mathbf{Z}} - \mathbf{Z}_t \ln(\mathbf{Z}_t) + \mathbf{Z}_t^5 \\
 &\underset{(def.)}{=} \sum_{t=1}^T -d_{\text{KL}}(\mathbf{Z}_t | \mathbf{R}_t \Phi_t^{\mathbf{Z}}) = -D_{\text{KL}}(\mathbf{Z} | \mathbf{R} \odot \Phi^{\mathbf{Z}})
 \end{aligned}$$

où d_{KL} désigne la divergence de Kullback-Leibler définie par exemple dans [9] [10], $\mathbf{R} \odot \Phi^{\mathbf{Z}}$ désigne le produit de Hadamard des vecteurs $\mathbf{R}$ et $\Phi^{\mathbf{Z}}$, i.e., $(\mathbf{R} \odot \Phi^{\mathbf{Z}})_t = \mathbf{R}_t \Phi_t^{\mathbf{Z}}$, et

$$D_{\text{KL}} : \begin{cases} \mathbb{R}^T & \longrightarrow \mathbb{R} \\ \mathbf{R} & \longmapsto \sum_{t=1}^T d_{\text{KL}}(\mathbf{Z}_t | \mathbf{R}_t \Phi_t^{\mathbf{Z}}) \end{cases}$$

Nous cherchons à maximiser cette quantité par rapport à $\mathbf{R}$, donc à minimiser D_{KL} par rapport à $\mathbf{R}$. La fonction D_{KL} étant additivement séparable, cela revient à maximiser chaque terme individuellement par rapport à $\mathbf{R}_t$. La condition d'optimalité du premier ordre, c'est-à-dire l'annulation

5. Equivalent de Stirling : $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \Rightarrow \ln(n!) \sim \ln(\sqrt{2\pi}) + \ln(\sqrt{n}) - n + n \ln(n) \sim n \ln(n) - n$. L'équivalent est applicable ici car $\mathbf{Z}_t$ est grand, de l'ordre de quelques milliers de cas

du gradient, donne la formule explicite :

$$\hat{R}_t^{MV} = \frac{Z_t}{\Phi_t^Z} = \frac{Z_t}{\sum_{s=1}^{\tau_\Phi} \phi_s Z_{t-s}}, \quad \forall t \in \{1, \dots, T\}. \quad (1)$$

Comme on peut le voir sur la Figure 2, cet estimateur, tracé en gris clair, est difficilement exploitable : il y a des fluctuations très rapides à l'échelle de quelques jours alors que le temps typique de l'épidémie est plutôt 25 jours, durée maximale durant laquelle un individu infecté est contagieux. Le comportement erratique reflète en réalité les erreurs et délais de report des comptes d'infection : très sous-estimés, voire nuls le week-end, qui ne reflètent pas la dynamique de l'épidémie. L'estimateur du maximum de vraisemblance ne donne pas une bonne estimation de R_t et nous ne pouvons pas nous y fier.

Moyenne a posteriori avec a priori Gamma

Afin de palier le manque de robustesse du maximum de vraisemblance aux erreurs de report, un estimateur bayésien a été proposé [2]. Nous rappelons que nous avons modélisé le nombre de nouveaux cas à la date t sachant les comptes passés via une distribution de Poisson :

$$\mathbb{P}(Z_t | Z_1, \dots, Z_{t-1}, R_t) = \frac{(R_t \Phi_t^Z)^{Z_t} \exp(-R_t \Phi_t^Z)}{Z_t!}$$

Afin d'obtenir une estimée de R_t dont les variations temporelles soient plus réalistes, c'est-à-dire plus régulières, il a été proposé dans [2] de réaliser l'estimation de R_t au temps t comme s'il était constant durant les τ derniers jours ce qui conduit à la vraisemblance

$$\mathbb{P}(Z_{t-\tau+1}, \dots, Z_t | Z_1, \dots, Z_{t-\tau}, R_{t,\tau}) = \prod_{s=t-\tau+1}^t \frac{(R_{s,\tau} \Phi_s^Z)^{Z_s} \exp(-R_{s,\tau} \Phi_s^Z)}{Z_s!}$$

où $R_{t,\tau}$ désigne le nombre de reproduction à la date t et supposé constant sur la période $t - \tau + 1, \dots, t$. Cette fois-ci au lieu d'une inconnue pour une observation nous avons une inconnue pour τ observations.

Dans un formalisme bayésien, une loi a priori Gamma de paramètres (a, b) , conjuguée à la loi de Poisson, est défini sur $R_{t,\tau}$. Cela permet notamment de ne pas sous-estimer le nombre de reproduction lorsque le nombre de cas est faible en début d'épidémie grâce à un prior dont la moyenne est très élevée (en pratique, la moyenne du prior Gamma est de 5, une valeur très grande devant un R typique de un). L'application de la loi de Bayes conduit à

$$\begin{aligned} & \mathbb{P}(Z_{t-\tau+1}, \dots, Z_t, R_{t,\tau} | Z_1, \dots, Z_{t-\tau}) \\ &= \mathbb{P}(Z_{t-\tau+1}, \dots, Z_t | Z_1, \dots, Z_{t-\tau}, R_{t,\tau}) \mathbb{P}(R_{t,\tau}) \\ &= \left(\prod_{s=t-\tau+1}^t \frac{(R_{s,\tau} \Phi_s^Z)^{Z_s} \exp(-R_{s,\tau} \Phi_s^Z)}{Z_s!} \right) \left(\frac{R_{t,\tau}^{a-1} \exp(-R_{t,\tau}/b)}{\Gamma(a) b^a} \right) \\ &= R_{t,\tau}^{a + \sum_{s=t-\tau+1}^t Z_s - 1} \exp \left(-R_{t,\tau} \left(\sum_{s=t-\tau+1}^t \Phi_s^Z + \frac{1}{b} \right) \right) \prod_{s=t-\tau+1}^t \frac{(\Phi_s^Z)^{Z_s}}{Z_s!} \cdot \frac{1}{\Gamma(a) b^a} \\ &\propto R_{t,\tau}^{a + \sum_{s=t-\tau+1}^t Z_s - 1} \exp \left(-R_{t,\tau} \left(\sum_{s=t-\tau+1}^t \Phi_s^Z + \frac{1}{b} \right) \right) \prod_{s=t-\tau+1}^t \frac{(\Phi_s^Z)^{Z_s}}{Z_s!} \end{aligned}$$

Ainsi, la distribution a posteriori de $\mathbf{R}_{t,\tau}$ est une distribution Gamma de paramètres

$$\left(a + \sum_{s=t-\tau+1}^t Z_s, \frac{1}{\frac{1}{b} + \sum_{s=t-\tau+1}^t \Phi_s^Z} \right)$$

Le résultat calculé avec $\tau = 15$ jours est tracé en vert sur la Figure 2. L'estimée varie de manière beaucoup plus continue, l'influence du bruit administratif, notamment les week-ends, est réduite : cela est plus réaliste que le maximum de vraisemblance. Il y a néanmoins toujours la présence de fluctuations résiduelles qui empêchent de lire des grandes tendances.

Estimateur variationnel avec régularisation temporelle

La mauvaise qualité des comptes de cas de COVID-19 a motivé la construction d'estimateurs plus robustes au bruit administratif grâce à une régularisation temporelle favorisant la linéarité par morceaux [4], [5]. En effet, une estimée de $\mathbf{R}_t$ linéaire par morceaux est facilement compréhensible et interprétable, et il est aisé de voir lorsque celle-ci va passer en dessous ou au dessus de un, qui est une valeur intéressante car elle permet de dire si l'épidémie va respectivement se réduire ou se propager de manière exponentielle. L'opposée de la log-vraisemblance est utilisée comme attache aux données, favorisant l'adéquation avec le modèle épidémiologique. Une pénalisation en norme ℓ_1 de la dérivée seconde va quant à elle imposer la parcimonie de la dérivée seconde et donc la linéarité par morceaux (Une fonction linéaire par morceaux est caractérisée par des dérivées secondes nulles). Notre objectif est alors de trouver $\hat{\mathbf{R}}$ tel que

$$\hat{\mathbf{R}} = \underset{\mathbf{R}}{\operatorname{argmin}} D_{\text{KL}}(\mathbf{Z}|\mathbf{R} \odot \Phi^Z) + \lambda \|\mathbf{D}_2 \mathbf{R}\|_1 \quad (2)$$

où $\mathbf{D}_2$ est l'opérateur de dérivée seconde discret normalisé :

$$\mathbf{D}_2 = \frac{1}{4} \begin{bmatrix} 1 & -2 & 1 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & & \ddots & \ddots & \vdots \\ 0 & \cdots & 1 & -2 & 1 & 0 \\ 0 & \cdots & 0 & 1 & -2 & 1 \end{bmatrix}$$

Nous appellerons par la suite $D_{\text{KL}}(\mathbf{Z}|\mathbf{R} \odot \Phi^Z) + \lambda \|\mathbf{D}_2 \mathbf{R}\|_1$ la fonction objectif. L'expression D_{KL} représente l'attache aux données, $\lambda \|\cdot\|_1$ représente la pénalisation (en norme 1). Dans notre cas,

- on fait l'hypothèse que $\Phi_t^Z \neq 0$, $\forall t$. Pour que Φ_t^Z soit nulle, il faudrait qu'aucun nouveau cas ne soit détecté pendant 25 jours consécutifs, ce qui correspondrait à la fin de l'épidémie.
- le terme d'attache aux données D_{KL} est strictement convexe.

La fonction objectif est strictement convexe, d'où l'existence d'un unique minimum. Le calcul de cet estimateur nécessite de résoudre ce problème d'optimisation convexe non lisse. L'estimateur obtenu pour une forte pénalisation ($\lambda = 50$) est tracé en orange sur la Figure 2. Nous obtenons bien une fonction linéaire par morceau, conforme à nos attentes : compréhensible, interprétable et il est aisé de voir lorsque ce dernier passe au dessus ou en dessous de un.

2.2 Résolution d'un problème d'optimisation convexe non lisse

Soient $\mathcal{X}$ et $\mathcal{Y}$ deux $\mathbb{R}$ -espaces vectoriels de dimension finie. On considère $F : \mathcal{Y} \rightarrow [0, +\infty[$ et $G : \mathcal{X} \rightarrow [0, +\infty[$ deux applications convexes, propres (ayant au moins une valeur finie, cf. Définition 2) et semi-continues inférieurement (ce qui équivaut à avoir un épigraphe fermé,

Algorithme 1 Algorithme de Chambolle-Pock pour la minimisation de (3)

- Initialisation : Choisir $\tau, \sigma > 0$, $\theta \in [0, 1]$, $(x^0, y^0) \in X \times Y$. On pose $\bar{x}^0 = x^0$.
- Itérations ($k \geq 0$) : mettre à jour $x^k, y^k, \bar{x}^k$ tels que :

$$\begin{cases} y^{k+1} = (Id + \sigma \partial F^*)^{-1}(y^k + \sigma K \bar{x}^k) \\ x^{k+1} = (Id + \tau \partial G)^{-1}(x^k - \tau K^* y^{k+1}) \\ \bar{x}^{k+1} = x^{k+1} + \theta(x^{k+1} - x^k) \end{cases}$$

cf. Définition 4). Enfin on considère $K : \mathcal{X} \longrightarrow \mathcal{Y}$ un opérateur linéaire continu. Pour résoudre le problème de minimisation général dont on suppose qu'il existe au moins une solution $(x, y) \in \mathcal{X} \times \mathcal{Y}$

$$\hat{x} \in \underset{x}{\operatorname{Argmin}} F(Kx) + G(x) \quad (3)$$

nous pouvons utiliser l'algorithme de Chambolle-Pock décrit par l'Algorithme 1 et proposé dans [11]. Cet algorithme utilise une généralisation du gradient : la sous-différentielle, notée ∂F , dont on pourra retrouver la définition en Définition 2. L'opérateur $(Id + \sigma \partial F)^{-1}$ s'écrit de manière alternative sous forme d'un opérateur proximal. L'opérateur proximal d'une fonction f est défini par (Voir Annexe C pour plus de détail) :

$$\begin{aligned} \operatorname{prox}_{\gamma f} : \mathcal{H} &\longrightarrow \mathcal{H} \\ x &\longmapsto \operatorname{armin}_{y \in \mathcal{H}} f(y) + \frac{1}{2\gamma} \|x - y\|_2^2 \end{aligned}$$

De plus nous savons que

$$p = \operatorname{prox}_f(x) \iff x \in (Id + \partial f)(p) \iff p(x) = (Id + \partial f)^{-1}(x)$$

Cela nous permet donc de reformuler l'algorithme en termes d'opérateurs proximaux (on pourra retrouver la définition de tels opérateurs ainsi que quelques propriétés dans l'Annexe C). La structure de l'algorithme permet donc de résoudre des problèmes où F n'est pas différentiable. On

Terme	Symbole	Description
G	$D_{\text{KL}}(\mathbf{Z} \cdot \odot \Phi^{\mathbf{Z}})$	divergence KL
F	$\lambda \ \cdot\ _1$	régularisation ℓ_1
K	D_2	opérateur de différences finies du second ordre
x^k	$\mathbf{R}^{[k]}$	variable primale
y^k	$\mathbf{Q}^{[k]}$	variable duale
$\bar{x}^k$	$\bar{\mathbf{R}}^{[k]}$	variable auxiliaire

TABLE 1 – Variables et opérateurs utilisés dans la formulation primal-duale.

utilise alors l'Algorithme 2 [5], [4] dont les principales variables et les opérateurs sont listés dans la Table 1. Les estimateurs obtenus pour différents paramètres de régularisation et pour $k_{\max} = 10^6$, $\epsilon = 10^{-8}$, $\sigma = \tau = \frac{0.99}{\|D_2\|_{\text{op}}}$ sont représentés Figure 4. Par ailleurs, pour des fonctions F et G convexes, propres, semi-continues inférieurement, si $F(K) + G$ admet au moins un minimum alors l'algorithme de Chambolle-Pock converge vers un minimiseur. Si la fonctionnelle est de plus strictement convexe, alors ce minimiseur est unique, et l'initialisation n'influe pas sur la convergence de

Algorithme 2 Minimisation primal-duale de la divergence Kullback-Leibler pénalisée

Entrées : Comptes d'infections $\mathbf{Z} \in \mathbb{R}^T$, infectiosité globale $\Phi^{\mathbf{Z}} \in \mathbb{R}^T$, précision ϵ , nombre maximal d'itération $k_{\max}$

Choisir des paramètres $\tau, \sigma > 0$ tels que $\sigma\tau\|\mathbf{D}_2\|_{\text{op}}^2 < 1$

Initialisation : $\mathbf{R}^{[0]} = \hat{\mathbf{R}}^{MV}$, $\mathbf{Q}^{[0]} = \mathbf{D}_2\mathbf{R}^{[0]}$, $\bar{\mathbf{R}}^{[0]} = \mathbf{R}^{[0]}$

$k \leftarrow 0$

Tant que $k < k_{\max}$ et $\|\mathbf{R}^{[k]} - \mathbf{R}^{[k+1]}\|_2 > \epsilon$ **faire :**

$$\mathbf{Q}^{[k+1]} = \text{prox}_{\sigma(\lambda\|\cdot\|_1)^*} \left(\mathbf{Q}^{[k]} + \sigma\mathbf{D}_2\bar{\mathbf{R}}^{[k]} \right)$$

$$\mathbf{R}^{[k+1]} = \text{prox}_{\tau\mathbf{D}_{\text{KL}}(\mathbf{Z}|\cdot\odot\Phi^{\mathbf{Z}})}(\mathbf{R}^{[k]} - \tau\mathbf{D}_2^{\top}\mathbf{Q}^{[k+1]})$$

$$\bar{\mathbf{R}}^{[k+1]} = 2\mathbf{R}^{[k+1]} - \mathbf{R}^{[k]}$$

$k \leftarrow k + 1$

Fin

l'algorithme. Nous ferons donc toujours le choix d'initialiser avec le résultat donné par le maximum de la log-vraisemblance. En outre, on a

$$\mathbf{R}^{[0]} = \bar{\mathbf{R}}^{[0]} = \mathbf{Z}/\Phi^{\mathbf{Z}}$$

et on initialise donc $\mathbf{Q}^{[0]} = \mathbf{D}_2\mathbf{R}^{[0]}$.

Dans le cas d'une régularisation faible, correspondant à $\lambda = 0, 5$ en bleu sur la Figure 4, l'estimateur se révèle nettement moins sensible aux bruits administratifs que celui basé sur un a priori Gamma, ce qui le rend plus proche de la réalité que les deux estimateurs précédents. Toutefois, il présente un nombre élevé de ruptures de pente. A l'inverse, avec une régularisation forte, correspondant à $\lambda = 50$ en orange sur la Figure 4, l'estimateur reste peu sensible aux bruits administratifs tout en affichant beaucoup moins de ruptures de pente que dans le cas faiblement régularisé. Nous privilégions donc l'estimateur à forte régularisation ($\lambda = 50$), car celui obtenu avec une régularisation faible ($\lambda = 0, 5$) produit des ruptures trop fréquentes et rapprochées. Or, un tel comportement est peu réaliste d'un point de vue épidémiologique, l'évolution de l'épidémie ne pouvant pas varier aussi rapidement en raison de la période de contagiosité, fixée ici à $\tau_{\Phi} = 25$ jours. Il est à noter cependant que pour un faible niveau de régularisation, (en bleu sur la Figure 4), la précision ϵ sur l'accroissement de $\mathbf{R}$ a été atteinte après environ 6×10^3 itérations. En comparaison pour un fort niveau de régularisation, (en orange sur la Figure 4), il a fallu environ 6×10^5 itérations, ce qui est presque 100 fois plus ! De manière générale, plus nous augmentons le paramètre de régularisation plus le nombre d'itérations nécessaire pour arriver à convergence augmente.

Afin d'estimer le nombre de reproduction, nous avons dû introduire des paramètres de modèle lors de la construction du modèle : l'opérateur linéaire $\mathbf{D}_2$ ainsi que le paramètre de pénalisation λ . Ces paramètres vont modifier la forme et le niveau de régularisation temporelle de $\mathbf{R}_t$. De plus, pour résoudre le problème d'optimisation sous-jacent à l'estimation de $\mathbf{R}_t$, nous avons introduit des paramètres algorithmiques σ et τ . Ces paramètres vont quant à eux modifier la vitesse de convergence.

2.3 Apprentissage statistique et réseaux de neurones déroulés

Notre objectif est d'estimer une quantité d'intérêt y à partir d'une observation x , en cherchant à minimiser l'erreur d'estimation. Comme la véritable erreur n'est pas directement accessible, on

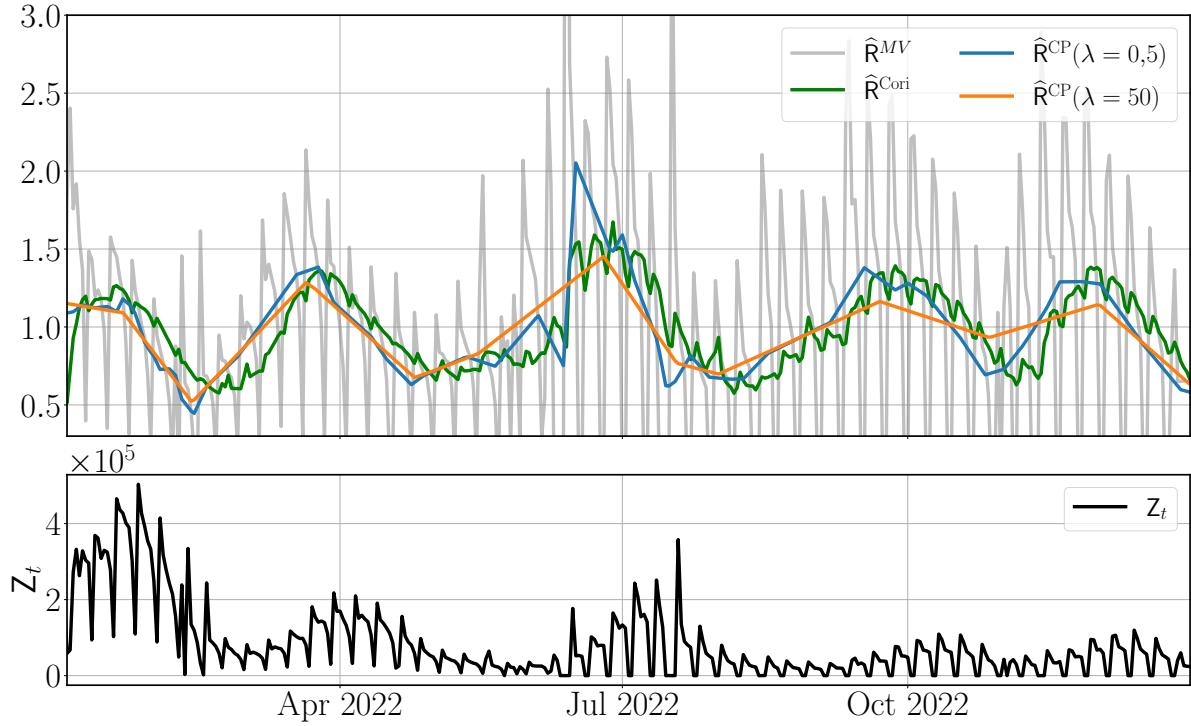


FIGURE 4 – Première ligne : estimées du nombre de reproduction obtenues par minimisation de la fonction objectif (2) via l’Algorithme de Chambolle-Pock 2 pour une pénalisation faible ($\lambda = 0,5$ en bleu) et une pénalisation forte ($\lambda = 50$ en orange). *En vert* : moyenne a posteriori pour un modèle bayésien avec vraisemblance de Poisson et une loi a priori Gamma et régularisation temporelle. *En gris* : estimateur du maximum de vraisemblance. Deuxième ligne : décompte journalier des nouveaux cas Z_t .

utilise une base de données annotée $\{(x_k, y_k), k = 1, \dots, n\}$ et l’on remplace cette erreur par une version empirique, mesurée à l’aide d’une fonction de perte $\mathcal{L}$. L’apprentissage statistique consiste alors à construire un estimateur $\hat{f}_\Theta$, dépendant de paramètres ajustables Θ , et à déterminer ceux-ci en minimisant l’erreur empirique :

$$\Theta^\dagger \in \underset{\Theta}{\operatorname{Argmin}} \frac{1}{n} \sum_{k=1}^n \mathcal{L}(\hat{f}_\Theta(x_k), y_k), \quad (4)$$

Cette démarche permet de sélectionner les paramètres de sorte que l’estimateur s’adapte au mieux aux données observées.

Parmi les estimateurs paramétriques employés dans ce cadre, les réseaux de neurones à propagation avant (ou *feedforward neural networks*) constituent une classe particulièrement répandue. Ces modèles sont composés de couches *successives*⁶ de neurones artificiels, chaque couche réalisant une opération affine puis une transformation non linéaire des données à l’aide d’une fonction d’activation. Analytiquement, dans la couche $k + 1$, la relation entre l’entrée $\mathbf{x}^{[k]}$ et la sortie $\mathbf{x}^{[k+1]}$ est :

$$\mathbf{x}^{[k+1]} = \eta^{[k]}(\mathbf{W}^{[k]}\mathbf{x}^{[k]} + b^{[k]}) \quad (5)$$

6. Un tel réseau de contient pas de couches qui réinjecteraient l’information des couches de surfaces vers les couches profondes

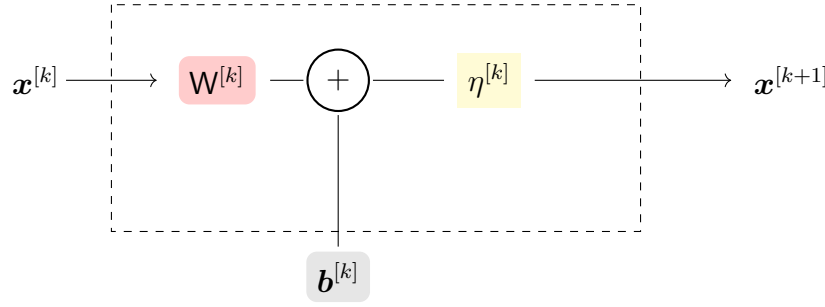


FIGURE 5 – Représentation d'une couche d'un réseau à propagation avant : W est une application linéaire, b est un biais et η est une fonction d'activation.

où W est une application linéaire, b est un vecteur de biais, et η est la fonction d'activation appliquée composante par composante. Un réseau à propagation avant de K couches est alors représenté par

$$f_{\Theta}(\mathbf{x}) = \eta^{[K]} \left(W^{[K]} \dots \eta^{[1]} \left(W^{[1]} \mathbf{x} + b^{[1]} \right) \dots + b^{[K]} \right) \quad (6)$$

où Θ représente les paramètres du modèle : $\Theta = (W^{[1]}, \dots, W^{[K]}, b^{[1]}, \dots, b^{[K]})$. Une couche est schématisée en Figure 5.

Deux fonctions d'activation classiques, représentée Figure 6 sont :

- la tangente hyperbolique : $\eta(x) = \tanh(x)$.
- la fonction ReLU (*Rectified Linear Unit*) : $\eta(x) = \max(0, x)$.

L'entraînement du réseau consiste à ajuster les paramètres de toutes les couches en minimisant la fonction de perte globale (4) via des algorithmes d'optimisation, généralement une variante de la descente de gradient comme **Adam** (voir [12]) ou **SGD** (voir [13]). La méthode de dépliement

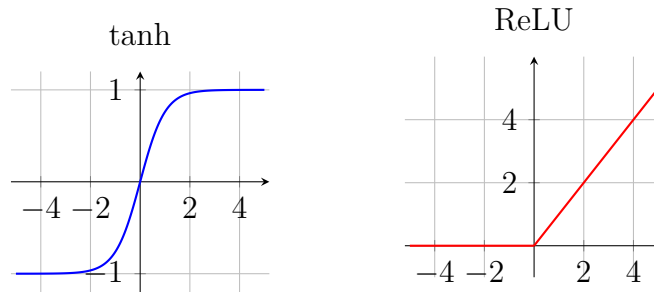


FIGURE 6 – Fonctions d'activation classiques : tangente hyperbolique et ReLU.

d'algorithme sous forme de réseau de neurones, initiée dans [14] puis rapidement généralisée à de nombreux algorithmes de traitement du signal et des images [6], établit un lien entre algorithmes itératifs et réseaux de neurones à propagation avant et permet d'obtenir des architectures interprétables et efficaces. Prenons un exemple proche de notre problème d'intérêt : la représentation parcimonieuse. La représentation parcimonieuse des signaux constitue un enjeu central et récurrent en traitement du signal. On pose couramment le problème comme suit : étant donné un vecteur $\mathbf{y} \in \mathbb{R}^n$ et un dictionnaire surcomplet $W \in \mathbb{R}^{n \times m}$ où $m > n$, on cherche une représentation parcimonieuse de $\mathbf{y}$ à partir de W . En d'autres termes, on cherche une représentation $\mathbf{x} \in \mathbb{R}^m$ telle que $\mathbf{y} \simeq W\mathbf{x}$ en encourageant un grand nombre de coefficients de $\mathbf{x}$ à être nuls. Une approche bien

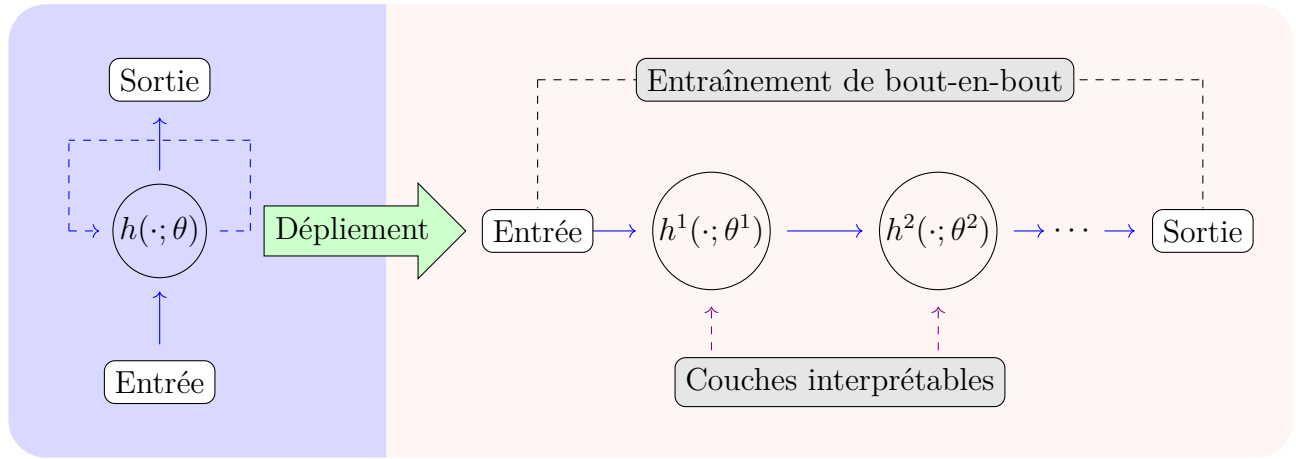


FIGURE 7 – Schéma de dépliement d'un algorithme itératif.
Figure inspirée de [6, Figure 1].

connue pour déterminer $\mathbf{x}$, appelée le *lasso* [15], consiste à régulariser la norme ℓ_1 du vecteur $\mathbf{x}$. Nous obtenons alors le problème d'optimisation convexe suivant :

$$\min_{\mathbf{x} \in \mathbb{R}^m} \frac{1}{2} \|\mathbf{y} - \mathbf{W}\mathbf{x}\|_2^2 + \lambda \|\mathbf{x}\|_1 \quad (7)$$

où $\lambda > 0$ est un paramètre de régularisation qui contrôle la parcimonie de $\mathbf{x}$. Nous pouvons dès à présent établir une analogie entre les équations (2) et (7) : dans (2), l'attache aux données est donnée par la divergence de Kullback-Leibler D_{KL} , tandis que dans (7) elle correspond à une perte quadratique. De plus, la pénalisation porte respectivement sur $D_2 \mathbf{R}$ dans (2) et sur $\mathbf{x}$ dans (7). Pour résoudre (7), une méthode bien connue proposée dans [16] est Iterative Shrinkage and Thresholding Algorithm (ISTA), que nous avons introduit à l'Algorithme 3.

Algorithme 3 Iterative Shrinkage and Thresholding Algorithm permettant de résoudre le problème (7)

Initialisation $\mathbf{x}^{[0]}$, pas de descente $\mu > 0$, précision ϵ

Tant que $\|\mathbf{x}^{[k+1]} - \mathbf{x}^{[k]}\| > \epsilon$ **faire** :

$$\mathbf{x}^{[k+1]} = h_\lambda \left(\left(\mathbf{I} - \frac{1}{\mu} \mathbf{W}^\top \mathbf{W} \right) \mathbf{x}^{[k]} + \frac{1}{\mu} \mathbf{W}^\top \mathbf{y} \right)$$

Fin

La fonction h_λ est appelée soft thresholding (ou seuillage doux), dont l'expression est :

$$h_\lambda(x) = \begin{cases} \lambda & \text{si } x > \lambda \\ -\lambda & \text{si } x < -\lambda \\ x & \text{sinon} \end{cases}$$

Remarquons que cette application non linéaire, tracée en rouge sur la Figure 9a, ressemble beaucoup à la fonction d'activation Tanh, en bleu. Cette observation a motivé le dépliement de l'algorithme ISTA sous forme d'un réseau de neurones.

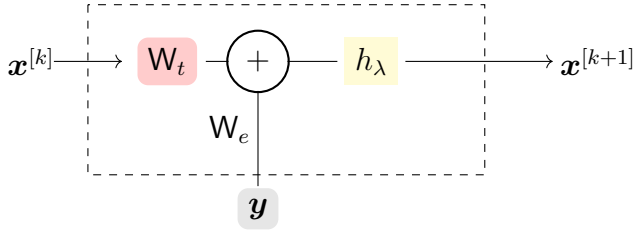


FIGURE 8 – Représentation d'une couche de ISTA déroulé

$$\mathbf{x}^{[k+1]} = \eta^{[k]} (\mathbf{W}^{[k]} \mathbf{x}^{[k]} + b^{[k]})$$

où

$$\begin{cases} \eta^{[k]} = h_\lambda \\ \mathbf{W}^{[k]} = \mathbf{W}_t \\ b^{[k]} = \mathbf{W}_e \mathbf{y} \end{cases}$$

Le dépliement d'algorithme est une technique développée dans [14] : c'est le fait de considérer chaque itération d'un algorithme itératif (tel que ISTA) comme une couche d'un réseau de neurones comme représenté Figure 7. Si l'on prend l'algorithme ISTA, en faisant la substitution $\mathbf{W}_t = \mathbf{I} - \frac{1}{\mu} \mathbf{W}^\top \mathbf{W}$ et $\mathbf{W}_e = \frac{1}{\mu} \mathbf{W}^\top$ on peut considérer chaque itération de ISTA comme une couche d'un réseau de neurones, dont la relation entre l'entrée $\mathbf{x}^{[k]}$ et la sortie $\mathbf{x}^{[k+1]}$ de la couche pourrait être représentée par l'Equation (5). La couche résultante est représentée à la Figure 8. Nous pouvons remarquer que contrairement à l'algorithme itératif 3 qui itère jusqu'à atteindre une précision ϵ , le réseau de neurones comporte un nombre K de couches fixé.

Le réseau déroulé est entraîné sur un jeu de données pour optimiser les paramètres $\mathbf{W}_t$, $\mathbf{W}_e$ et λ . L'optimisation se passe comme dans le cadre d'apprentissage statistique décrit en début de cette section. La version apprise, LISTA, peut atteindre une efficacité supérieure à celle d'ISTA : en particulier elle permet d'atteindre un niveau de précision donnée 20 fois plus rapidement que ISTA comme présenté dans [14]. Elle est également utile lorsque le dictionnaire $\mathbf{W}$ n'est pas connu. En plus de pouvoir apprendre les paramètres, le dépliement permet de considérer des fonctions et des paramètres spécifiques à chaque couche, offrant une plus grande flexibilité que les schémas itératifs. Par exemple sur la Figure 7, on voit que l'on va appliquer la fonction h paramétrée par θ un certain nombre de fois lorsque l'on utilise un algorithme itératif (partie gauche du schéma). Lorsque l'on déplie l'algorithme (partie droite du schéma), chaque couche a sa propre fonction avec sa propre paramétrisation. Ainsi à la couche k , on appliquera la fonction h^k avec les paramètres θ^k au résultat provenant de la couche $k - 1$. Pour l'algorithme ISTA (cf. Algorithme 3), on pourrait avoir

$$\mathbf{x}^{[k+1]} = h_\lambda^{[k]} \left(\mathbf{W}_t^{[k]} \mathbf{x}^{[k]} + \mathbf{W}_e^{[k]} \mathbf{y} \right)$$

où les termes en rouge peuvent être appris, la dépendance en k mentionnant leur spécificité à chaque couche.

3 Estimation du nombre de reproduction via un algorithme proximal déroulé

Nous présentons à présent les contributions originales de ce stage.

3.1 Déroulement de l'algorithme de Chambolle-Pock

Cette section décrit en détail le dépliement du schéma algorithmique de Chambolle-Pock pour l'estimation du nombre de reproduction décrit à l'Algorithme 2. L'originalité majeure de ce travail

est d'avoir conservé l'attache aux données poissonniennes durant l'intégralité du dépliement de l'algorithme ce qui se traduit par la présence d'une fonction d'activation non conventionnelle dans le réseau résultant. Cette démarche contraste avec la pratique usuelle qui se ramène à une attache aux données gaussiennes, comme proposé dans [17] par exemple. Chaque itération de l'Algorithme 2, correspondant au système d'équations suivant

$$\begin{cases} \mathbf{Q}^{[k+1]} = \text{prox}_{\sigma(\lambda\|\cdot\|_1)^*} \left(\mathbf{Q}^{[k]} + \sigma \mathbf{D}_2 \bar{\mathbf{R}}^{[k]} \right) \\ \mathbf{R}^{[k+1]} = \text{prox}_{\tau \mathbf{D}_{\text{KL}}(\mathbf{Z} \cdot \odot \Phi \mathbf{Z})} (\mathbf{R}^{[k]} - \tau \mathbf{D}_2^\top \mathbf{Q}^{[k+1]}) \\ \bar{\mathbf{R}}^{[k+1]} = 2\mathbf{R}^{[k+1]} - \mathbf{R}^{[k]} \end{cases} \quad (8)$$

est interprétée comme une couche d'un réseau de neurones à propagation avant prenant en entrée $\mathbf{R}^{[k]}$, $\bar{\mathbf{R}}^{[k]}$ et $\mathbf{Q}^{[k]}$ et renvoyant en sortie $\mathbf{R}^{[k+1]}$, $\bar{\mathbf{R}}^{[k+1]}$ et $\mathbf{Q}^{[k+1]}$. L'objectif est d'écrire cette suite d'opérations sous la forme d'une succession d'opérations élémentaires de la forme présentée à l'équation (5) et illustrée en Figure 5.

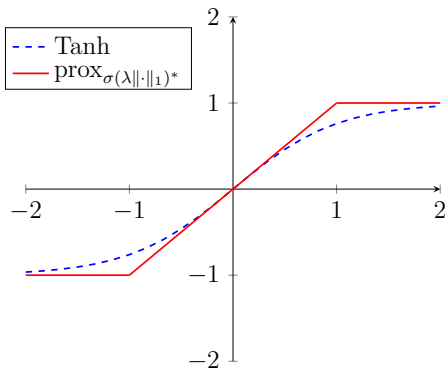
Dans un premier temps nous réécrivons l'étape de mise à jour de $\mathbf{Q}^{[k+1]}$ en terme de fonction d'activation, d'opération linéaire et de biais. Nous observons que :

$$\begin{pmatrix} \mathbf{0}_{T-2,T} & \mathbf{D}_2 & \sigma^{-1} \mathbf{I}_{T-2} \end{pmatrix} \begin{pmatrix} \mathbf{R}^{[k]} \\ \bar{\mathbf{R}}^{[k]} \\ \mathbf{Q}^{[k]} \end{pmatrix} = \mathbf{D}_2 \bar{\mathbf{R}}^{[k]} + \sigma^{-1} \mathbf{Q}^{[k]}$$

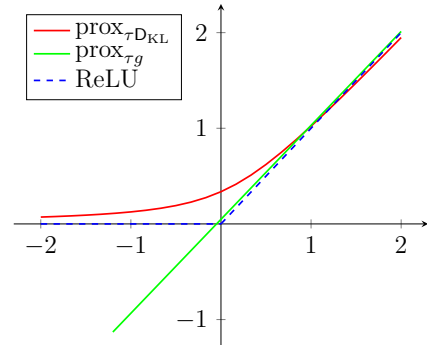
puis en appliquant la fonction $\text{prox}_{\sigma(\lambda\|\cdot\|_1)^*}$ on retrouve l'expression de $\mathbf{Q}^{[k+1]}$. Or

$$\text{prox}_{\sigma(\lambda\|\cdot\|_1)^*}(x) = \begin{cases} \lambda\sigma^{-1} & \text{si } x > \lambda\sigma^{-1} \\ -\lambda\sigma^{-1} & \text{si } x < -\lambda\sigma^{-1} \\ x & \text{sinon} \end{cases}$$

Nous retrouvons l'expression de la fonction $\text{HardTanh}_{\lambda\sigma^{-1}}$, que nous conserverons pour la suite. Grâce à la Figure 9a, nous pouvons constater que cet opérateur ressemble à une fonction d'activation plus commune (voir Section 2.3) : la fonction tangente hyperbolique.



(a) Opérateur $\text{prox}_{\sigma(\lambda\|\cdot\|_1)^*}$ et comparaison à la fonction d'activation Tanh.



(b) Opérateur $\text{prox}_{\tau \mathbf{D}_{\text{KL}}}$, opérateur $\text{prox}_{\tau g}$ et comparaison à la fonction d'activation ReLU.

FIGURE 9 – Analogie entre fonctions d'activation usuelles et les opérateurs proximaux.

Le calcul de $\mathbf{R}^{[k+1]}$ nécessitant à la fois $\mathbf{Q}^{[k+1]}$ et $\mathbf{R}^{[k]}$ et le réseau étant strictement à propagation

avant nous proposons de transmettre intégralement le triplet $(\mathbf{R}^{[k]}, \bar{\mathbf{R}}^{[k+1]}, \mathbf{Q}^{[k+1]})$ au travers de cette première étape, ce qui se traduit par :

$$\mathbf{W}_1^{[k]} = \begin{pmatrix} \mathbf{I}_T & \mathbf{0}_{T,T} & \mathbf{0}_{T,T-2} \\ \mathbf{0}_{T,T} & \mathbf{I}_T & \mathbf{0}_{T,T-2} \\ \mathbf{0}_{T-2,T} & \mathbf{D}_2 & \sigma^{-1}\mathbf{I}_{T-2} \end{pmatrix} \quad \eta_1^{[k]} = \begin{Bmatrix} Id \\ Id \\ \sigma \text{Hardtanh}_{\lambda\sigma^{-1}} \end{Bmatrix} \quad \mathbf{b}_1^{[k]} = 0 \quad (9)$$

$$\implies \eta_1^{[k]}(\mathbf{W}_1^{[k]} \begin{pmatrix} \mathbf{R}^{[k]} \\ \bar{\mathbf{R}}^{[k]} \\ \mathbf{Q}^{[k]} \end{pmatrix}) = \eta_1^{[k]} \left(\begin{pmatrix} \mathbf{R}^{[k]} \\ \bar{\mathbf{R}}^{[k]} \\ \mathbf{Q}^{[k]} + \sigma \mathbf{D}_2 \bar{\mathbf{R}}^{[k]} \end{pmatrix} \right) = \begin{pmatrix} \mathbf{R}^{[k]} \\ \bar{\mathbf{R}}^{[k]} \\ \mathbf{Q}^{[k+1]} \end{pmatrix}$$

Forts de nos résultats, nous pouvons à présent exprimer la mise à jour de $\mathbf{R}^{[k]}$, ligne 2 de l'Equation (8), sous la forme d'une couche neuronale élémentaire. Nous observons que

$$\begin{pmatrix} \mathbf{I}_T & \mathbf{0}_T & -\tau \mathbf{D}_2^\top \end{pmatrix} \begin{pmatrix} \mathbf{R}^{[k]} \\ \bar{\mathbf{R}}^{[k]} \\ \mathbf{Q}^{[k+1]} \end{pmatrix} = \mathbf{R}^{[k]} - \tau \mathbf{D}_2^* \mathbf{Q}^{[k+1]}$$

Ainsi, en appliquant la fonction

$$\text{prox}_{\tau \mathbf{D}_{\text{KL}}(\mathbf{Z}|\cdot \odot \Phi^{\mathbf{Z}})} = \frac{\cdot - \tau \Phi_t^{\mathbf{Z}} + \sqrt{(\cdot - \tau \Phi_t^{\mathbf{Z}})^2 + 4\tau \mathbf{Z}_t}}{2}$$

dont le calcul est détaillé dans l'Annexe C Exemple 1, nous obtenons l'expression de $\mathbf{R}^{[k+1]}$. La fonction d'activation $\text{prox}_{\tau \mathbf{D}_{\text{KL}}(\mathbf{Z}|\cdot \odot \Phi^{\mathbf{Z}})}$ peut paraître assez inhabituelle, mais il se trouve qu'elle ressemble à la fonction ReLU comme illustré en Figure 9b.

Si des fonctions ressemblantes ont déjà été utilisées⁷, l'originalité de ce travail est de considérer une fonction d'activation exactement égale à l'opérateur proximal de la divergence de Kullback-Leibler, reflétant l'attache aux données poissonnienne du problème (2). La stratégie habituelle est de se ramener à une attache aux données gaussiennes, comme proposé dans [17]. L'opérateur proximal d'une attache aux données gaussienne est défini par

$$\text{prox}_{\tau g}(x) = \frac{\tau + x}{1 + \tau/\mathbf{Z}} \quad (\text{voir calcul Annexe C, Exemple 2})$$

qui est une fonction linéaire en la variable d'intérêt. Notre architecture aura donc une non-linéarité supplémentaire, permettant d'être plus discriminant tout en restant proche du modèle épidémiologique. Cet opérateur est également tracé sur la Figure 9b à titre de comparaison. Comme précédemment nous souhaitons conserver certaines données, ce qui implique des modifications :

$$\mathbf{W}_2^{[k]} = \begin{pmatrix} \mathbf{I}_T & \mathbf{0}_T & -\tau \mathbf{D}_2^\top \\ \mathbf{0}_{T,T-2} & \mathbf{I}_T & \mathbf{0}_T \\ \mathbf{0}_{T-2,T} & \mathbf{0}_T & \mathbf{I}_T \\ \mathbf{I}_{T-2} & \mathbf{0}_T & \mathbf{0}_T \end{pmatrix} \quad \eta_2^{[k]} = \begin{Bmatrix} \text{prox}_{\tau \mathbf{D}_{\text{KL}}(\mathbf{Z}|\cdot \odot \Phi^{\mathbf{Z}})} \\ Id \\ Id \\ Id \end{Bmatrix} \quad \mathbf{b}_2^{[k]} = 0 \quad (10)$$

$$\implies \eta_2^{[k]}(\mathbf{W}_2^{[k]} \begin{pmatrix} \mathbf{R}^{[k]} \\ \bar{\mathbf{R}}^{[k]} \\ \mathbf{Q}^{[k+1]} \end{pmatrix}) = \eta_2^{[k]} \left(\begin{pmatrix} \mathbf{R}^{[k]} - \tau \mathbf{D}_2^\top \mathbf{Q}^{[k+1]} \\ \mathbf{R}^{[k]} \\ \bar{\mathbf{R}}^{[k]} \\ \mathbf{Q}^{[k+1]} \end{pmatrix} \right) = \begin{pmatrix} \mathbf{R}^{[k+1]} \\ \mathbf{R}^{[k]} \\ \bar{\mathbf{R}}^{[k]} \\ \mathbf{Q}^{[k+1]} \end{pmatrix}$$

7. On retrouve l'étude d'un opérateur proximal similaire appelé « bent identity » dans [18, Exemple 2.8]

Enfin, il reste à exprimer la mise à jour de $\bar{\mathbf{R}}^{[k+1]}$, ligne 3 de l'Equation (8). Nous observons que :

$$\begin{pmatrix} 2\mathbf{I}_T & -\mathbf{I}_T & \mathbf{0}_T & \mathbf{0}_{T,T-2} \end{pmatrix} \begin{pmatrix} \mathbf{R}^{[k+1]} \\ \mathbf{R}^{[k]} \\ \bar{\mathbf{R}}^{[k]} \\ \mathbf{Q}^{[k+1]} \end{pmatrix} = \bar{\mathbf{R}}^{[k+1]}$$

Nous pouvons appliquer la fonction Id pour rester dans le cadre défini par l'équation (6). Enfin, pour transmettre les données, nous faisons le changement suivant :

$$\begin{aligned} \mathbf{W}_3^{[k]} &= \begin{pmatrix} \mathbf{I}_T & \mathbf{0}_T & \mathbf{0}_T & \mathbf{0}_{T,T-2} \\ 2\mathbf{I}_T & -\mathbf{I}_T & \mathbf{0}_T & \mathbf{0}_{T,T-2} \\ \mathbf{0}_{T-2,T} & \mathbf{0}_{T-2,T} & \mathbf{0}_{T-2,T} & \mathbf{I}_{T-2} \end{pmatrix} \quad \eta_3^{[k]} = \begin{Bmatrix} Id \\ Id \\ Id \end{Bmatrix} \quad \mathbf{b}_3^{[k]} = 0 \\ \implies \eta_3^{[k]}(\mathbf{W}_3^{[k]} \begin{pmatrix} \mathbf{R}^{[k+1]} \\ \mathbf{R}^{[k]} \\ \bar{\mathbf{R}}^{[k]} \\ \mathbf{Q}^{[k+1]} \end{pmatrix}) &= \begin{pmatrix} \mathbf{R}^{[k+1]} \\ \bar{\mathbf{R}}^{[k+1]} \\ \mathbf{Q}^{[k+1]} \end{pmatrix} \end{aligned} \quad (11)$$

Remarque 1 *L'une des contributions dans le dépliement de l'algorithme de Chambolle-Pock est l'utilisation de l'opérateur proximal de la divergence de Kullback-Leibler pour une distribution de Poisson : de nombreux travaux de la littérature, bien qu'ayant un modèle poissonnien, se ramènent à une attache aux données gaussienne comme présenté dans [17] par exemple. En effet, soit X suivant une loi de Poisson de paramètre $\alpha > 0$. Alors le théorème central limite donne*

$$\frac{X - \alpha}{\sqrt{\alpha}} \xrightarrow{\alpha \rightarrow \infty} \mathcal{N}(0, 1)$$

Autrement dit, pour α assez grand, on peut faire l'approximation X suit une loi normale $\mathcal{N}(\alpha, \alpha)$. Ainsi lorsque l'on considère la variable aléatoire Z_t :

$$\alpha = \mathbf{R}_t \Phi_t^Z = \mathbf{R}_t \sum_{s=1}^{\tau_\Phi} \phi_s Z_{t-s}$$

Or lorsque l'épidémie est active, $\mathbf{R}_t$ est de l'ordre de grandeur de 1, et $\sum_{s=1}^{\tau_\Phi} \phi_s Z_{t-s}$ est une moyenne pondérée de Z donc du même ordre de grandeur de Z . Ainsi $\alpha \propto Z_t$ et on peut donc faire l'approximation

$$Z_t \hookrightarrow \mathcal{N}(\mathbf{R}_t \Phi_t^Z, \mathbf{R}_t \Phi_t^Z).$$

Dans notre situation, nous ne faisons pas cette approximation, car les ordres de grandeur varient significativement entre les vagues épidémiologiques et les creux dus au bruit administratif. Nous préférons donc une attache aux données exacte.

Un des avantage majeur des algorithmes dépliés est leur plus grande flexibilité par rapport à l'algorithme itératif sous-jacent : nous pouvons considérer des paramètres et des fonctions différentes à chaque couche, représenté par exemple par les $h^k(\cdot; \theta^k)$ à la couche k lors du dépliement Figure 7. Dans notre cas, à la couche $k + 1$, il est possible de déplier la k -ième itération de l'Algorithme de Chambolle-Pock 2 de manière très général en :

$$\begin{cases} \mathbf{Q}^{[k+1]} = \text{prox}_{\sigma^{[k]}(\lambda^{[k]} \|\cdot\|_1)^*} \left(\mathbf{Q}^{[k]} + \sigma^{[k]} \mathbf{M}^{[k]} \bar{\mathbf{R}}^{[k]} \right) \\ \mathbf{R}^{[k+1]} = \text{prox}_{\tau^{[k]} \text{D}_{\text{KL}}(\mathbf{Z} | \cdot \odot \Phi \mathbf{Z})} (\mathbf{R}^{[k]} - \tau^{[k]} \mathbf{M}^{[k] \top} \mathbf{Q}^{[k+1]}) \\ \bar{\mathbf{R}}^{[k+1]} = 2\mathbf{R}^{[k+1]} - \mathbf{R}^{[k]} \end{cases} \quad (12)$$

grâce à la généralisation suivante des applications linéaires et fonctions d'activations définies précédemment :

$$\begin{aligned}
 W_1^{[k]} &= \begin{pmatrix} \mathbf{I}_T & \mathbf{0}_{T,T} & \mathbf{0}_{T,T-2} \\ \mathbf{0}_{T,T} & \mathbf{I}_T & \mathbf{0}_{T,T-2} \\ \mathbf{0}_{T-2,T} & \mathbf{M}^{[k]} & \sigma^{[k]-1} \mathbf{I}_{T-2} \end{pmatrix} & \eta_1^{[k]} &= \begin{pmatrix} Id \\ Id \\ \sigma^{[k]} \text{Hardtanh}_{\lambda^{[k]} \sigma^{[k]} - 1} \end{pmatrix} \\
 W_2^{[k]} &= \begin{pmatrix} \mathbf{I}_T & \mathbf{0}_T & -\tau^{[k]} \mathbf{M}^{[k]\top} \\ \mathbf{0}_{T,T-2} & \mathbf{I}_T & \mathbf{0}_T \\ \mathbf{0}_{T,T-2} & \mathbf{0}_T & \mathbf{I}_T \\ \mathbf{I}_{T-2} & \mathbf{0}_T & \mathbf{0}_T \end{pmatrix} & \eta_2^{[k]} &= \begin{pmatrix} \text{prox}_{\tau^{[k]} D_{\text{KL}}}(\mathbf{Z} | \cdot \odot \Phi^{\mathbf{Z}}) \\ Id \\ Id \\ Id \end{pmatrix} \\
 W_3^{[k]} &= \begin{pmatrix} \mathbf{I}_T & \mathbf{0}_T & \mathbf{0}_T & \mathbf{0}_{T,T-2} \\ 2\mathbf{I}_T & -\mathbf{I}_T & \mathbf{0}_T & \mathbf{0}_{T,T-2} \\ \mathbf{0}_{T-2,T} & \mathbf{0}_{T-2,T} & \mathbf{0}_{T-2,T} & \mathbf{I}_{T-2} \end{pmatrix} & \eta_3^{[k]} &= \begin{pmatrix} Id \\ Id \\ Id \end{pmatrix}.
 \end{aligned}$$

Pour résumer, la $k+1$ -ième couche paramétrée par la collection de paramètres $\theta_k = (\lambda^{[k]}, \sigma^{[k]}, \tau^{[k]}, \mathbf{M}^{[k]})$ du réseau est représentée Figure 10.

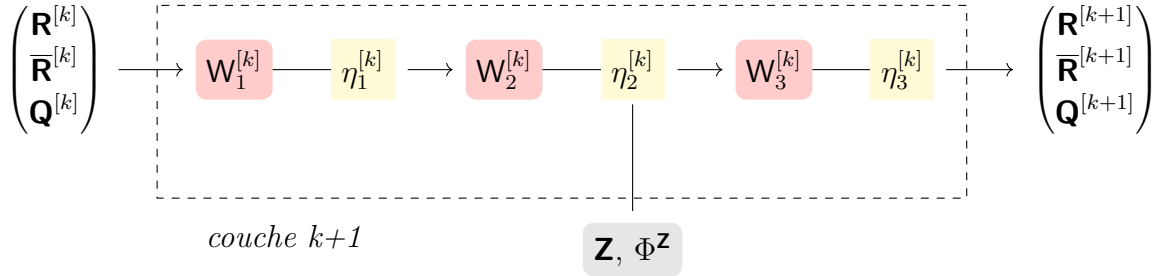


FIGURE 10 – Couche $k + 1$ du réseau à propagation avant.

L'architecture ainsi conçue constitue une structure complexe, déduite du modèle. Ceci donne de l'interprétabilité aux réseaux déroulés, contrairement à la majorité des réseaux profonds.

Nous initialiserons le réseau de la même manière que l'algorithme de Chambolle-Pock, ce qui termine le dépliement de notre algorithme. Un schéma global récapitulatif du réseau ainsi construit est donné Figure 11, où chaque couche reprend la structure déroulée schématisée à la Figure 10. Nous noterons la sortie de ce réseau de neurones à

$\mathbf{Z}$, et donc a priori à $\Phi^{\mathbf{Z}}$, fixés $\mathcal{NN}_{\Theta}(\mathbf{Z})$ où $\Theta = (\theta_k)_{k=1,\dots,K}$.

3.2 Evaluation des capacités d'approximation du réseau déroulé

Pour chaque série temporelle $\mathbf{Z}$, des experts ont aidé à fournir un λ^* qui fournisse une estimée la plus réaliste possible. Par exemple dans le cas de la France, les experts ont validé l'estimée pour $\lambda^* = 50$ tracé en rouge sur la Figure 12. Nous allons dans un premier temps essayer de reproduire cet estimateur validé par les experts à partir de notre réseau de neurones.

A titre de première illustration, nous proposons de sélectionner les paramètres du réseau exactement comme ceux utilisés dans 2, puis de comparer la sortie du réseau, correspondant ainsi à K itérations de l'Algorithme 2, à l'estimée obtenue à convergence. Nous fixons alors : $\mathbf{M}^{[k]} = \mathbf{D}_2$, $\sigma^{[k]} = \tau^{[k]} = 0.99 / \|\mathbf{D}\|_2$, $\lambda^{[k]} = 50$ et $k_{\text{layer}} = 500$. Nous appliquons ensuite ce réseau non entraîné aux données épidémiologiques mesurées en France, dont le résultat est représenté Figure

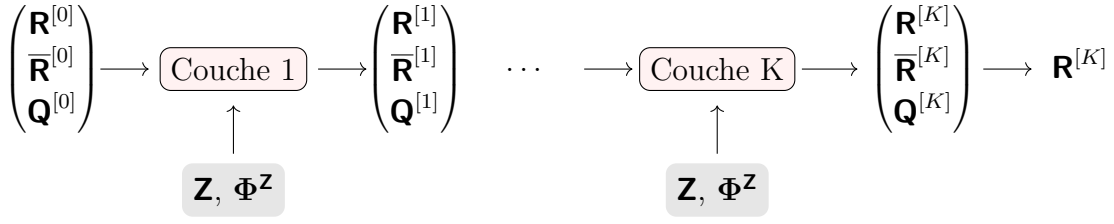


FIGURE 11 – Schéma global du déroulement de l'algorithme de Chambolle-Pock.

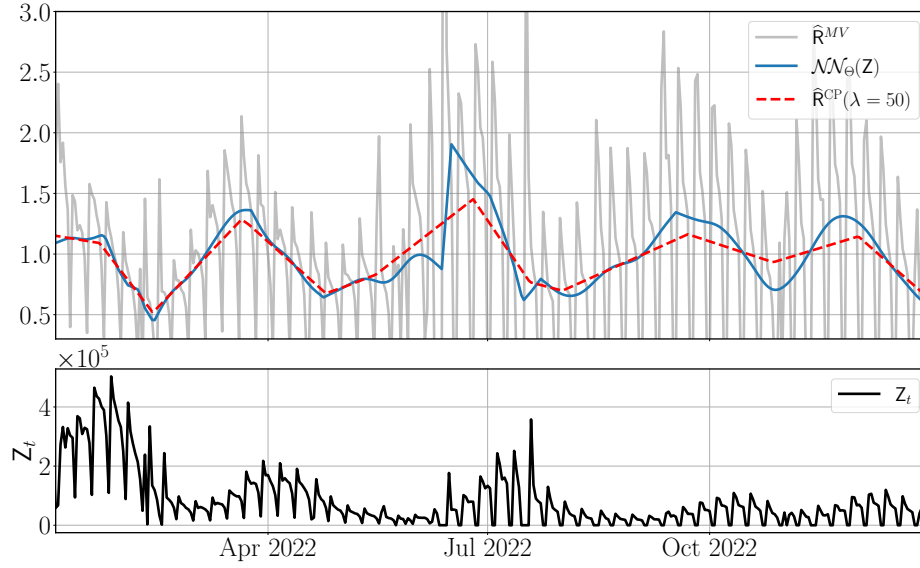


FIGURE 12 – Première ligne : *En bleu* : inférence du réseau sans apprentissage. *En rouge* : estimateur obtenu avec l'algorithme de Chambolle-Pock et une pénalisation forte ($\lambda = 50$). *En gris* : l'estimateur du maximum de vraisemblance. Deuxième ligne : décompte journalier des nouveaux cas Z_t .

12 en bleu. La courbe varie certes continûment, mais pas de manière linéaire par morceaux. En effet, sans entraînement et avec le choix de paramètres effectué, l'inférence du réseau non entraîné est simplement $K = 500$ itérations de l'algorithme de Chambolle-Pock. Cependant nous avons vu précédemment que pour avoir des fonctions linéaires par morceaux, il fallait beaucoup plus d'itérations pour que l'algorithme converge. Il est donc nécessaire d'ajuster les paramètres, et une question se pose : jusqu'à quel point pouvons-nous, en ajustant les paramètres Θ , s'approcher de l'estimée réaliste du nombre de reproduction $\hat{R}^{CP}(\lambda = 50)$, tracée en rouge sur la Figure 12 ?

3.2.1 Stratégie d'apprentissage s'appuyant sur un oracle

Afin de tester les capacités d'approximation du réseau, nous allons dans un premier temps considérer un oracle, noté $\mathbf{R}^*$, et ajuster les poids Θ afin de se rapprocher le plus possible de cet oracle. Pour apprendre les paramètres, nous allons utiliser la stratégie suivante :

- Nous nous fixons un λ^* afin de calculer $\mathbf{R}^*$ solution de l'équation (2) avec le schéma de Chambolle-Pock décrit par l'Algorithme 2, en fixant $k_{\max} = 1e^6$, $\epsilon = 1e^{-8}$, $\tau = \sigma = 0.99 / \|\mathbf{D}_2\|$, $\mathbf{M} = \mathbf{D}_2$ et en prenant les données $\mathbf{Z}$ et Φ^Z de la France. Ce $\mathbf{R}^*$ sera notre oracle, représenté en rouge sur le Figure 12

- Nous créons un réseau de neurones de $K = 500$ couches, composées de paramètres Θ apprenables. Ce nombre de couches est a priori un bon compromis entre le faible nombre de couches habituel en apprentissage statistique et le nombre important de couches nécessaire à l'algorithme de Chambolle-Pock pour converger.
- Nous résolvons le problème d'optimisation (E) :

$$\Theta^\dagger \in \underset{\Theta}{\operatorname{Argmin}} \frac{1}{T} \|\mathcal{NN}_\Theta(\mathbf{Z}) - \mathbf{R}^*\|_2^2 \quad (\text{E})$$

avec $\mathbf{R}^{[0]} = \bar{\mathbf{R}}^{[0]} = \mathbf{R}^{MV}$ et $\mathbf{Q}^{[0]} = \mathbf{D}_2 \bar{\mathbf{R}}^{[0]}$ à l'aide d'une descente de gradient (plus précisément Adam, voir Annexe B)

3.2.2 Ajustement du paramètre de régularisation

Nous avons une grande flexibilité concernant les poids apprenables Θ du réseau déplié. Le cas le plus simple est d'apprendre le paramètre de régularisation seul. Autrement dit nous avons $\Theta = \{\lambda\}$, et nous cherchons la solution du problème (E2) :

$$\lambda^\dagger \in \underset{\lambda}{\operatorname{Argmin}} \frac{1}{T} \|\mathcal{NN}_\lambda(\mathbf{Z}) - \mathbf{R}^*\|_2^2 \quad (\text{E2})$$

L'apprentissage se fera sur 5000 *epochs*.

Niveau de régularisation faible de l'oracle. Pour commencer, nous testons notre réseau déroulé sur un oracle trop faiblement régularisé (ici $\lambda = 0,5$ en bleu sur la Figure 4) pour lequel l'algorithme de Chambolle-Pock converge rapidement sur des faibles valeurs de λ . L'estimation obtenue et l'évolution de la perte E2 sont respectivement représentées sur la Figure 13a, première ligne et troisième ligne. La perte diminue de manière continue jusqu'à se stabiliser à une très faible valeur, ce qui traduit la convergence de la procédure d'apprentissage. L'oracle est reproduit, ce qui signifie que l'ajustement du seul paramètre de régularisation permet au réseau d'approximer très précisément des estimées faiblement régularisées. En effet, l'estimateur $\mathcal{NN}_{\lambda^\dagger}(\mathbf{Z})$ colle parfaitement à l'oracle $\mathbf{R}^*$ ce qui se traduit par une perte qui atteint presque 0. De plus cette perte s'est stabilisée asymptotiquement, ce qui caractérise la convergence du réseau. La valeur finale de lambda est $\lambda^\dagger = 0,49$, très proche de $\lambda^* = 0,5$. Cela montre que le réseau peut s'adapter à des oracles faiblement régularisés, tout en parvenant à retrouver avec précision le paramètre de régularisation sous-jacent, ce qui témoigne de sa forte similitude avec l'algorithme itératif de Chambolle-Pock 2.

Niveau de régularisation optimal de l'oracle. Nous testons à présent notre algorithme sur un oracle convenablement régularisé ($\lambda = 50$ en orange sur la Figure 4). L'estimation obtenue et l'évolution de la perte sont respectivement représentées sur la Figure 13b, première ligne et troisième ligne. Nous pouvons observer que la convergence est très lente, et le résultat est très différent de l'oracle visé. Nous pouvons donc nous demander si 5000 *epochs* sont suffisantes pour effectuer l'optimisation. Pour vérifier si la tâche d'apprentissage de λ via (E2) est correctement réalisée nous menons l'expérience suivante : pour une grille régulière de cent valeurs de λ logarithmiquement réparties entre 10^{-2} et 10^3 et pour chacun des oracles (faiblement régularisé ou correctement régularisé), nous évaluons la fonction de perte $\lambda \mapsto \frac{1}{T} \|\mathcal{NN}_\lambda(\mathbf{Z}) - \mathbf{R}^*\|_2^2$. La courbe obtenue est tracée en bleu sur la quatrième ligne de la Figure 13. Pour un oracle faiblement régularisé (Figure 13a) il y a un net minimum global, localisé autour de la valeur $\lambda = 0,5$ et atteignant une perte proche de zéro. Ainsi si nous initialisons le problème (E2) à $\lambda_{\text{init}} = 1$ (représenté par la croix verte sur

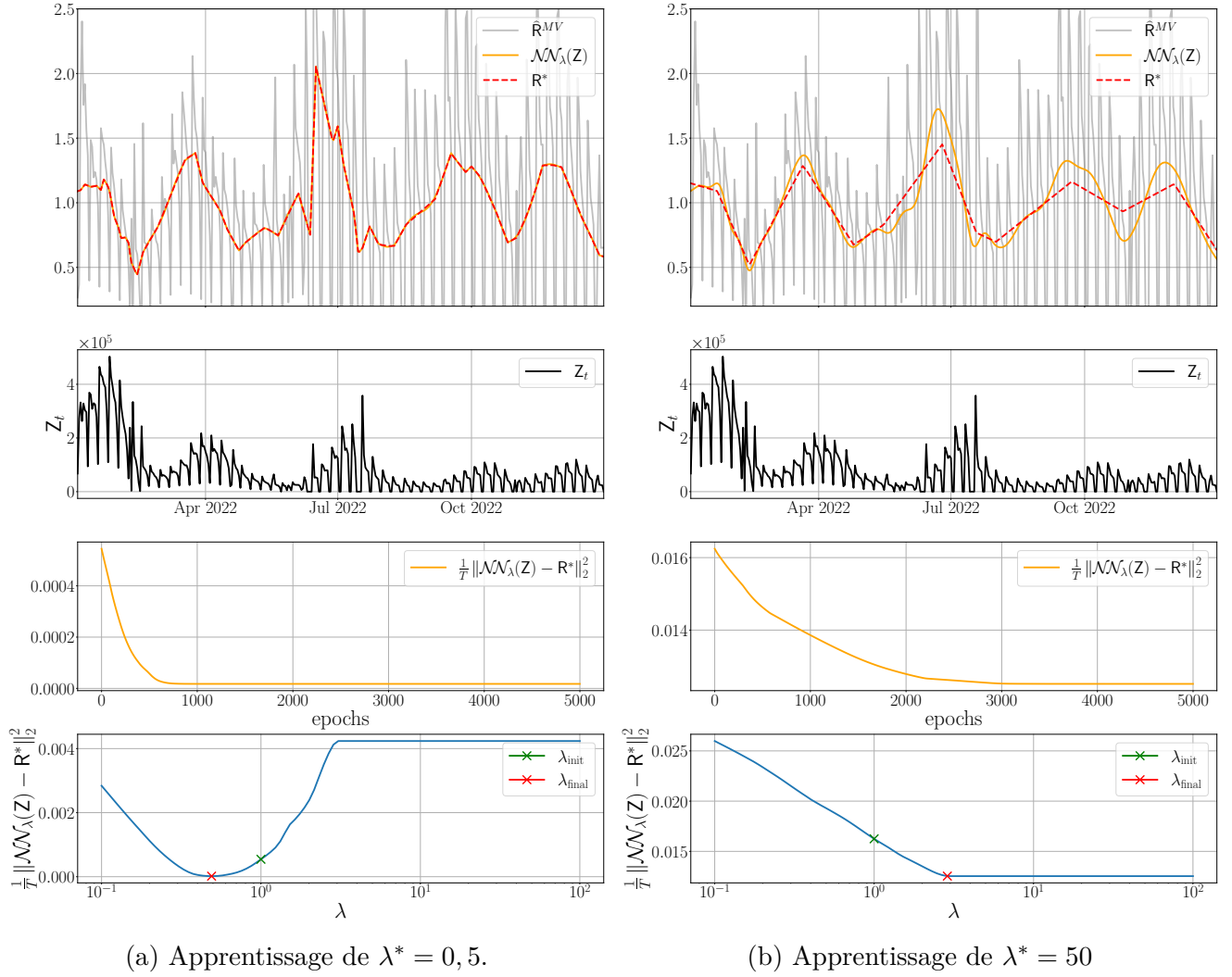


FIGURE 13 – Apprentissage du paramètre de régularisation. Première ligne : estimateur de R_t . Deuxième ligne : nouveau cas journalier Z_t . Troisième ligne : perte au cours des *epochs* lors de l'apprentissage. Quatrième ligne : perte pour différentes valeurs de λ .

la Figure 13a quatrième ligne) nous aboutissons bien au minimum global caractérisé par la croix rouge. L'estimateur est bien fidèle à l'oracle (cf. Figure 13a première ligne). Pour un oracle fortement régularisé, nous remarquons qu'il y a une zone très plate dans la fonction de perte comme illustré sur la Figure 13a : l'algorithme ne converge pas mais se bloque quelque part sur le plateau comme illustré sur la quatrième ligne de la Figure 13b.

La Figure 14 montre que pour un nombre d'itérations fixe (ici $\#iter = k_{layer} = 500$), il existe une valeur critique λ_c (que nous ne cherchons pas à calculer) telle que $\forall \lambda \geq \lambda_c, \mathcal{NN}_\lambda(\mathbf{Z}) = \mathcal{NN}_{\lambda_c}(\mathbf{Z})$. Ainsi à partir de ce λ_c , λ n'a plus d'influence et l'estimée reste donc inchangé, ce explique le palier Figure 13b quatrième ligne. L'allure étant bien différente de l'oracle $\mathbf{R}^*$, la perte stabilisée à une valeur non-nulle.

Nous en concluons qu'il ne sera pas possible d'obtenir des estimées du nombre de reproduction suffisamment régularisées avec l'apprentissage du seul paramètre de régularisation λ via (E2).

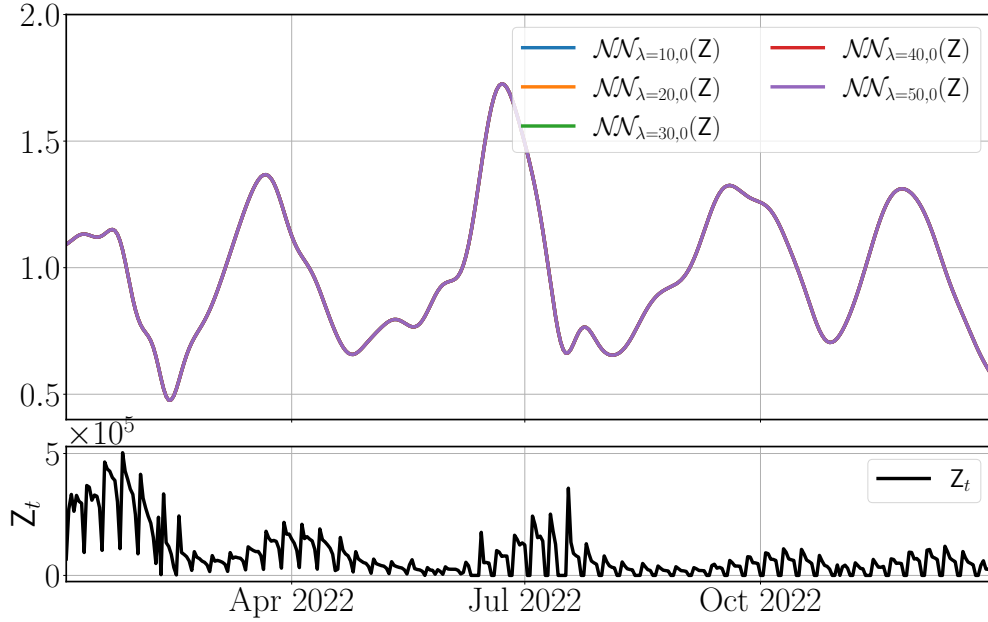


FIGURE 14 – Première ligne : inférence du nombre de reproduction via le réseau de neurones déplié paramétré par l'unique λ . Deuxième ligne : décompte journalier des nouveaux cas Z_t .

3.3 Capacités d'approximation de plusieurs jeux de poids

3.3.1 Comparaison de l'apprentissage de plusieurs jeux de poids

Afin de construire un réseau capable d'approximer des estimées linéaires par morceaux suffisamment régularisées pour être réalistes du point de vue épidémiologique, nous allons tirer parti de la flexibilité des réseaux déroulés afin d'augmenter le nombre de poids appris tout en conservant le caractère interprétable de l'architecture. Il faut cependant se placer au point critique entre conserver l'architecture venant du modèle et avoir un réseau assez complexe, donc avec assez de paramètres, pour pouvoir s'ajuster au comportement linéaire par morceaux des oracles.

Nous avons cherché à apprendre plusieurs combinaisons de paramètres Θ :

- apprentissage du paramètre de régularisation $\Theta = \lambda$: $\dim(\Theta) = 1$ paramètre
- apprentissage d'un paramètre de régularisation par couche $\Theta = \underline{\lambda} = (\lambda_k)_{k=1,\dots,K}$: $\dim(\Theta) = 500$ paramètres
- apprentissage des paramètres algorithmiques $\underline{\sigma} = (\sigma_k)_{k=1,\dots,K}$ et $\underline{\tau} = (\tau)_{k=1,\dots,K}$ à chaque couche, et d'un paramètre de pénalisation par couche $\underline{\lambda} : \Theta = \{\underline{\lambda}, \underline{\sigma}, \underline{\tau}\}$ et $\dim(\Theta) = 1500$ paramètres
- apprentissage des paramètres algorithmiques $\underline{\sigma}$ et $\underline{\tau}$, ainsi qu'un opérateur linéaire $\mathbf{M}$ par couche sous forme de filtre de taille 3. Cela revient, comme présenté dans (12), à remplacer l'opérateur de dérivée seconde $\mathbf{D}_2$ par un opérateur de filtrage plus général qui varie d'une couche à l'autre tout en conservant la taille du filtre initial : $\Theta = \{\underline{\sigma}, \underline{\tau}, \mathbf{M}_{AB}\}$ et $\dim(\Theta) =$

2000 paramètres

$$\mathbf{M}_{\underline{AB}}^{[k]} = \begin{bmatrix} a^{[k]} & b^{[k]} & a^{[k]} & 0 & \cdots & 0 \\ 0 & a^{[k]} & b^{[k]} & a^{[k]} & 0 & \cdots \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & a^{[k]} & b^{[k]} & a^{[k]} & 0 \\ 0 & \cdots & \cdots & a^{[k]} & b^{[k]} & a^{[k]} \end{bmatrix}$$

- apprentissage des paramètres algorithmiques $\underline{\sigma}$ et $\underline{\tau}$, ainsi qu'un opérateur linéaire $\mathbf{M}$ par couche sous forme de filtre de taille 25 : $\Theta = \{\underline{\sigma}, \underline{\tau}, \mathbf{M}_{\underline{C}}\}$ et $\dim(\Theta) = 13500$ paramètres

$$\mathbf{M}_{\underline{C}}^{[k]} = \begin{bmatrix} c_1^{[k]} & \cdots & c_{25}^{[k]} & 0 & \cdots & 0 \\ 0 & c_1^{[k]} & \cdots & c_{25}^{[k]} & 0 & \cdots \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \end{bmatrix}$$

La taille du filtre est guidée par la constante de temps caractéristique de l'épidémie de la COVID-19 (voir Figure 3) fournie par la durée maximale de contagiosité de 25 jours.

- apprentissage de l'opérateur $\mathbf{M}$ sans lui imposer de structure, mais en le choisissant commun à toutes les couches : $\Theta = \Gamma$ et $\dim(\Theta) = 132495$ paramètres

$$\Gamma = \begin{bmatrix} \gamma_{1,1} & \gamma_{1,2} & \cdots & \gamma_{1,T} \\ \gamma_{2,1} & \gamma_{2,2} & \cdots & \gamma_{2,T} \\ \vdots & \vdots & \vdots & \vdots \\ \gamma_{T-3,1} & \cdots & \gamma_{T-3,T-1} & \gamma_{T-3,T} \\ \gamma_{T-2,1} & \cdots & \gamma_{T-2,T-1} & \gamma_{T-2,T} \end{bmatrix}$$

- apprentissage de l'opérateur $\mathbf{M}$ en entier ainsi que les paramètres algorithmiques $\underline{\sigma}$ et $\underline{\tau}$: $\Theta = \{\underline{\sigma}, \underline{\tau}, \Gamma\}$ et $\dim(\Theta) = 133495$ paramètres

Pour chaque combinaison de poids, nous ajustons Θ en résolvant le problème (E) avec une descente de gradient **Adam**. Nous avons représenté en Figure 15 les estimées obtenues avec les réseaux entraînés dans quatre configurations : apprentissage d'un paramètre de régularisation par couche en bleu, noté $\hat{\mathbf{R}}^{\text{vect}}(\underline{\lambda}) = \mathcal{NN}_{\underline{\lambda}}(\mathbf{Z})$; apprentissage des paramètres algorithmiques σ et τ à chaque couche ainsi qu'un opérateur linéaire sous la forme de filtre de taille 3 par couche en orange, noté $\hat{\mathbf{R}}^{\text{vect}}(\underline{\sigma}, \underline{\tau}, \mathbf{M}_{\underline{AB}}) = \mathcal{NN}_{\underline{\sigma}, \underline{\tau}, \mathbf{M}_{\underline{AB}}}(\mathbf{Z})$; apprentissage des paramètres algorithmiques σ et τ à chaque couche ainsi qu'un opérateur linéaire sous la forme de filtre de taille 25 par couche en vert, noté $\hat{\mathbf{R}}^{\text{vect}}(\underline{\sigma}, \underline{\tau}, \mathbf{M}_{\underline{C}}) = \mathcal{NN}_{\underline{\sigma}, \underline{\tau}, \mathbf{M}_{\underline{C}}}(\mathbf{Z})$; ainsi que l'apprentissage de l'opérateur linéaire en entier (cette fois l'opérateur est commun à chaque couche) en rose, noté $\hat{\mathbf{R}}^{\text{vect}}(\Gamma) = \mathcal{NN}_{\Gamma}(\mathbf{Z})$. Pour comparaison, nous avons tracé l'estimateur de vraisemblance (1) en gris, et l'oracle visé, en pointillé rouge. Nous remarquons alors via les courbes bleue et orange que l'apprentissage respectivement de 500 et 2000 paramètres est insuffisant pour obtenir un estimé proche de l'oracle. La courbe verte illustre que l'apprentissage de 13500 paramètres permet d'obtenir un estimé plutôt proche de l'oracle. Enfin la courbe rose montre que l'apprentissage de $\sim 1,3 \times 10^5$ paramètres donne un estimé linéaire par morceau, très proche de l'oracle. Afin de quantifier les performances de ces différentes méthodes, nous définissons une nouvelle métrique indépendante de la perte en norme ℓ_2 utilisée pour l'apprentissage des paramètres (E) :

$$\mathcal{M}(\mathbf{R}^*, \mathbf{R}) = \sum_{t=1}^T \frac{|R_t - R_t^*|}{R_t^*}$$

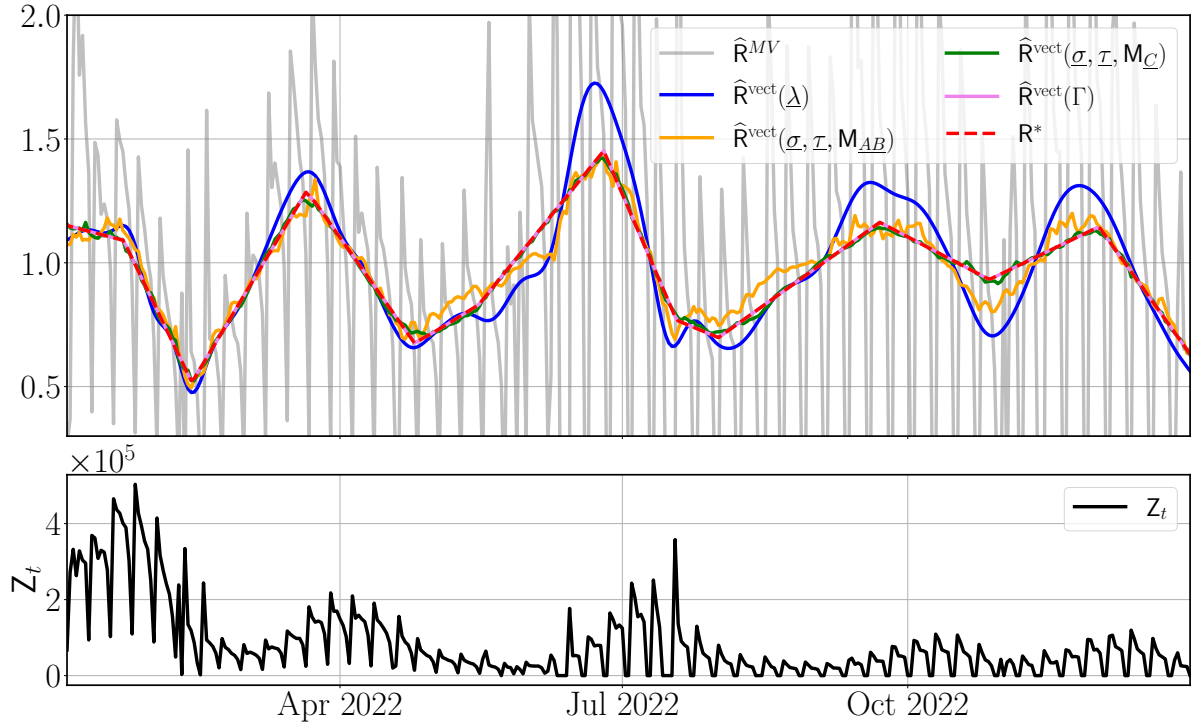


FIGURE 15 – Première ligne : estimateurs obtenus avec l'algorithme déroulé après l'apprentissage de différentes combinaisons de paramètres. Deuxième ligne : décompte journalier des nouveaux cas Z_t .

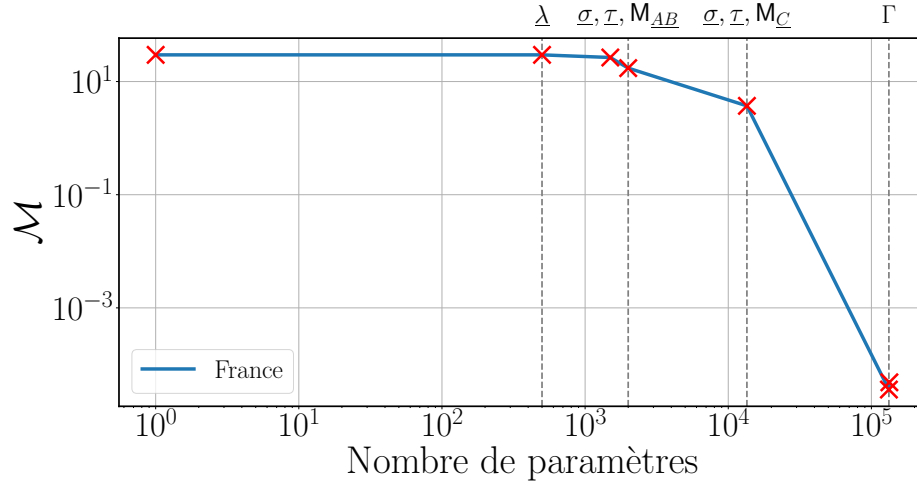


FIGURE 16 – Evolution de la qualité de l'apprentissage pour différentes combinaisons de paramètres.

Nous représentons sur la Figure 16 l'évolution de cette métrique en fonction du nombre de paramètres du réseau déroulé. Elle illustre que plus nous augmentons le nombre de paramètre, plus l'estimateur obtenu est proche de l'oracle.

Cette expérimentation permet de dégager des conclusions préliminaires. Tout d'abord, nous

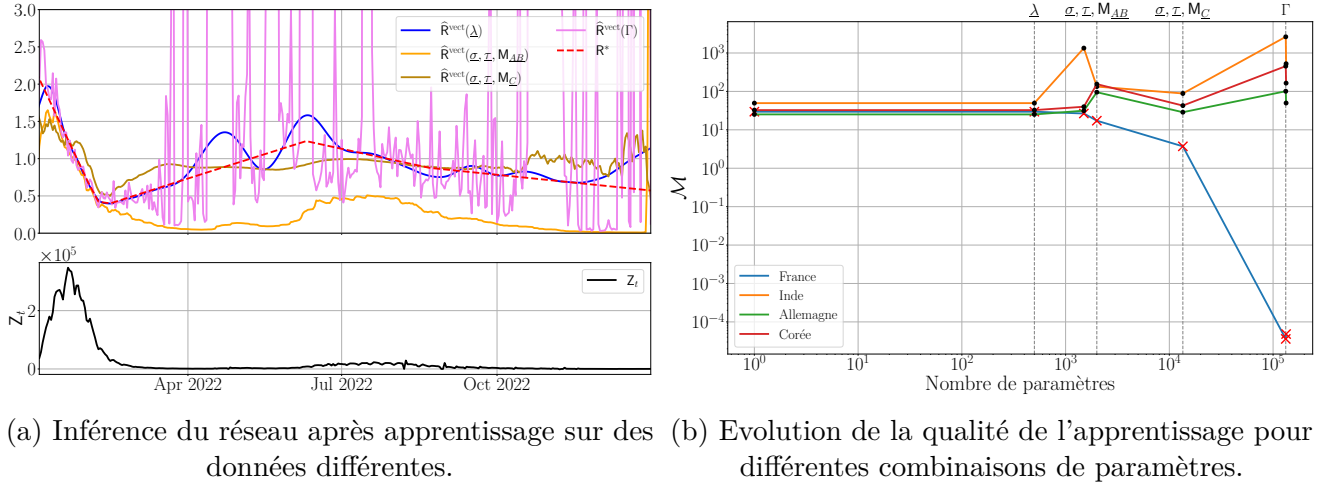


FIGURE 17 – Impact du sur-apprentissage.

sommes capables d’atteindre des estimateurs linéaires par morceaux avec l’algorithme déroulé. L’atout principal est qu’après l’apprentissage il suffit d’effectuer une inférence du réseau de 500 couches pour obtenir l’estimateur, contre plusieurs centaines de milliers d’itérations pour l’algorithme de Chambolle-Pock. Ensuite nous avons également montré qu’un nombre minimal de paramètres était nécessaire pour s’ajuster à un oracle suffisamment régularisé : de l’ordre de 10^4 pour un résultat moyen avec l’apprentissage de $\underline{\sigma}$, $\underline{\tau}$ et M_C , et de l’ordre de 10^5 avec l’apprentissage de l’opérateur linéaire entièrement. Dorénavant, nous savons également qu’il semble illusoire de faire des prédictions en apprenant $\Theta = \{\lambda\}$ ou $\Theta = \{\lambda, \underline{\sigma}, \underline{\tau}\}$ par exemple.

Nous avons vérifié que le réseau construit en Section 3.1, pourvu qu’il soit paramétré de manière suffisamment complexe (c’est-à-dire ayant suffisamment de poids apprenables), était capable de fournir des estimées linéaires par morceaux suffisamment régularisées lors de son entraînement à partir d’un oracle. Après avoir validé les capacités d’*approximation* du réseau, passons maintenant à l’exploration des capacités de *généralisation*. Afin d’estimer le nombre de reproduction dans plusieurs pays et à plusieurs périodes de temps simultanément, il est nécessaire de construire et entraîner un réseau qui possède un pouvoir de généralisation à des données nouvelles. Afin de tester le pouvoir de généralisation du réseau appris, nous allons procéder de la manière suivante : nous considérons les données de deux pays (par exemple la France et l’Inde) sur l’année 2022. Nous apprenons les paramètres du réseau sur les données françaises, ce qui permet d’obtenir $\Theta(\text{France})$. Nous effectuons ensuite une inférence du réseau : $\hat{\mathbf{R}}(\text{Inde}) = \mathcal{NN}_{\Theta(\text{France})}(\mathbf{Z}(\text{Inde}))$. Les résultats sont représentés Figure 17a. Sans surprise, nous observons très clairement un sur-apprentissage, qui était attendu car l’apprentissage se basait uniquement sur un oracle. Plus généralement, nous observons que plus nous augmentons le nombre de paramètres du réseau, plus la généralisation est mauvaise (voir Figure 17b). Il va donc falloir réaliser l’apprentissage des paramètres du réseau sur une base de données plus représentative.

3.3.2 Stratégie d’adaptation du pas d’apprentissage

L’apprentissage de jeux de poids comportant un grand nombre de paramètres peut être coûteux en temps de calcul : en effet, plus une couche contient de paramètres, plus chaque *epoch* est longue, et davantage d’*epochs* sont nécessaires pour converger, car le problème d’optimisation se déroule dans un espace de dimension plus élevée. Regardons l’évolution de la perte (E) au fur et à mesure

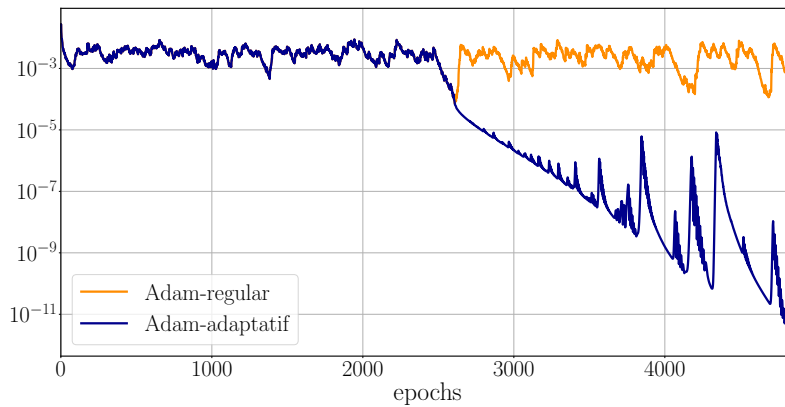


FIGURE 18 – Evolution de la perte (E) au fur et à mesure des *epochs* lors de l'apprentissage de Γ avec Adam. En orange : sans stratégie d'adaptation. En bleu : avec stratégie d'adaptation dépendamment des valeurs de la perte (E).

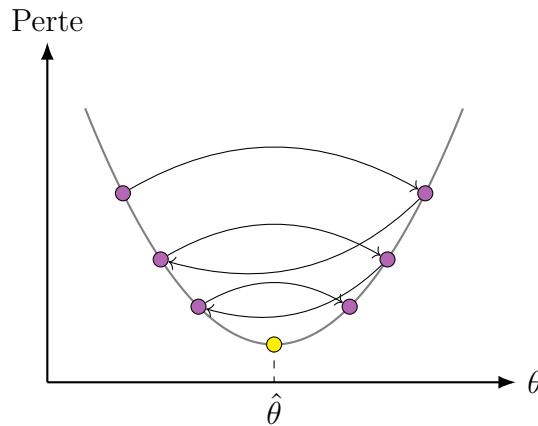


FIGURE 19 – Oscillations lors de la descente de gradient.
Figure inspirée de tex.stackexchange.com/questions/.

des *epochs* lors de l'apprentissage de Γ , tracée en orange à la Figure 18. Nous pouvons observer que la perte baisse puis qu'à partir d'environ 100 *epochs*, elle oscille (voire réaugmente significativement). Cela est dû au pas de descente qui est trop grand : comme on peut le voir sur la Figure 19 qui représente un exemple fictif en une dimension, avec un pas de descente trop grand, l'algorithme de minimisation génère une suite qui oscille autour du minimum sans réussir à l'atteindre. Une stratégie pour contourner cet écueil est d'ajuster le pas de descente au cours de l'apprentissage. Il est possible de changer le pas manuellement : nous avons proposé pour cela d'ajuster le pas de descente, noté η , à partir de 100 *epochs* car nous pouvons voir sur la Figure 18 en orange que c'est à partir de ce moment là que l'on oscille autour du minimum. Nous avons également proposé d'ajuster le pas de descente lorsque l'on franchit certains paliers de perte : par exemple $\eta = 10^{-4}$ si la perte passe en dessous de 10^{-4} , puis $\eta = 10^{-5}$ en dessous de 10^{-5} . Cette stratégie d'adaptation du pas de descente a permis d'obtenir les résultats de la Figure 15. L'évolution de la fonction de perte en utilisant la stratégie adaptative proposée est représentée en bleu en Figure 18.

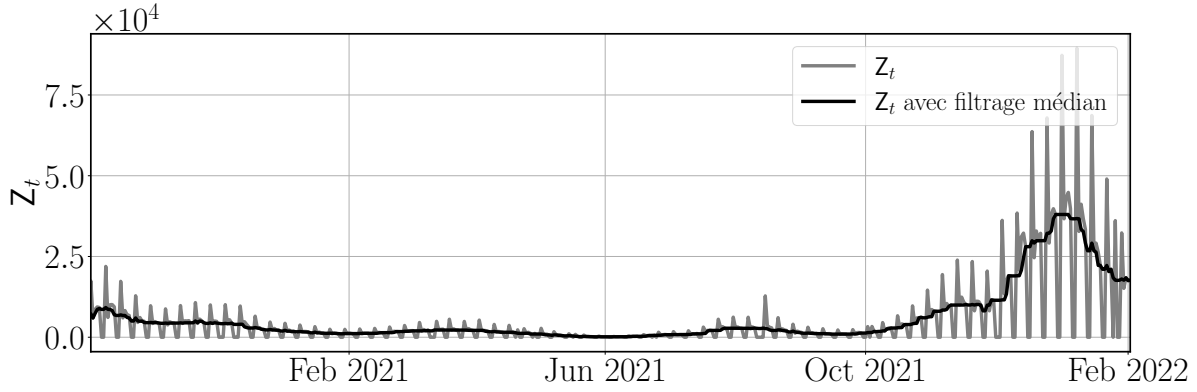


FIGURE 20 – Application d'un filtrage médian aux données suisses extraites de JHU.

4 Capacités de généralisation du réseau déroulé

Maintenant que la capacité du réseau à approximer un oracle a été validée, nous allons explorer ses capacités de généralisation. Pour cela une première étape est d'explorer la capacité du réseau entraîné sur une base de données de plusieurs exemples à approximer plusieurs oracles simultanément, puis une seconde étape consiste à tester l'architecture entraînée sur de nouvelles données. Pour ce faire, nous avons construit une base de données composée d'un ensemble de P séries temporelles $\mathbf{Z}^{(p)} = (Z_1, \dots, Z_T)$ correspondant aux cas de COVID-19 dans P pays, chacune associée à un oracle $\mathbf{R}^{(p)*}$ du nombre de reproduction effectif dans ce pays. L'entraînement consiste alors à résoudre le problème suivant :

$$\Theta^\dagger \in \underset{\Theta}{\operatorname{Argmin}} \frac{1}{P} \sum_{p=1}^P \frac{1}{T} \left\| \mathcal{NN}_\Theta(\mathbf{Z}^{(p)}) - \mathbf{R}^{(p)*} \right\|_2^2 \quad (\text{E3})$$

où $\mathcal{NN}_\Theta(\mathbf{Z}^{(p)})$ désigne la sortie du réseau de neurones pour le pays p , et $\mathbf{R}^{(p)*}$ l'oracle correspondant.

4.1 Construction de la base de données

Afin de construire une base de donnée représentative, nous prenons des observations $\mathbf{Z}^{(p)}$ de pays différents. Le choix des pays est fait de telle sorte à ce que nous ayons une diversité géographique et une diversité de comportement de la dynamique de l'épidémie : une vague ou plusieurs, d'intensités différentes (voir Annexe E). Chaque observation est associée à un oracle $\mathbf{R}^{(p)}$ à partir d'un niveau de régularisation équivalent à $\lambda^* = 50$ utilisé en Section 3 grâce à l'algorithme de Chambolle-Pock. Cependant comme on peut le voir sur la Figure 20 en gris, certains pays comportent beaucoup de jours où $Z_t = 0$. Sur la Figure 20 nous avons par exemple présenté les données suisses, comportant 151 valeurs de Z_t nulles. Ceci représente un niveau de bruit administratif extrêmement élevé et constitue une difficulté numérique majeure. Pour pallier ce problème, nous avons appliqué un pré-traitement par médian glissant aux données, inspiré de [4]. Si au jours t l'écart à la médiane est supérieur à 10% de la déviation absolue à la médiane sur une fenêtre τ , alors on fixe Z_t à la valeur médiane. Nous appliquons ici un filtrage médian de fenêtre $\tau = 15$ jours. On voit sur la Figure 20 en noir que cela permet d'enlever les nombreuses valeurs nulles tout en conservant la dynamique de $\mathbf{Z}$. Nous créons ainsi une base de données comportant la collection $\{(\mathbf{Z}^{(p)}, \mathbf{R}^{(p)*}), p = 1, \dots, P\}$.

4.2 Ajustement de l'architecture

Le passage à l'apprentissage sur une base de données implique également de changer l'architecture du réseau de neurones déroulé.

4.2.1 Profondeur du réseau déroulé

En pratique, l'entraînement d'un réseau de 500 couches sur une base de données composée de vingt couples de séries temporelles ($\mathbf{Z}, \mathbf{R}$) de longueur $T = 490$ jours s'est avéré trop coûteux : 5 000 *epochs*, qui sont grandement insuffisantes pour l'entraînement du réseau sur la base de données, ont nécessité près de 174 minutes de calcul. Dans [19] ou [20] qui mettent en oeuvre des techniques de dépliement d'algorithme, une vingtaine de couches sont utilisées. Nous pouvons également trouver dans [21] un résultat qui préconise que la profondeur du réseau soit telle que

$$k_{\text{layer}} \sim \frac{\log(k)}{-\log(\varrho_k)}$$

où ϱ_k est la vitesse de convergence de l'algorithme utilisé pour le dépliement et k est un entier suffisamment grand. Dans [11], il est démontré que $\varrho_k = \mathcal{O}(1/k)$. Nous proposons alors de réduire le nombre de couches à 10 couches. Afin de vérifier que cette modification n'altère pas les capacités d'approximation mises en évidence en Section 3, nous reprenons les simulations présentées en 3.3.1 en ne changeant que le nombre de couches, fixé à $k_{\text{layer}} = 10$. L'estimateur donné par l'apprentissage de l'opérateur linéaire Γ , en rose sur la Figure 21, illustre que l'oracle est estimé avec une précision comparable, mais obtenu cette fois en seulement 25 minutes. Nous avons également tracé en vert l'estimateur obtenu après apprentissage de $\underline{\sigma}$, $\underline{\tau}$ et $\mathbf{M}_{\underline{C}}$. Cet estimateur est nettement moins performant lorsque le réseau ne comporte que 10 couches, ce qui est cohérent puisque le nombre de paramètres de cette architecture dépend du nombre de couches. L'estimateur obtenu n'est plus assez réaliste, et nous nous concentrerons donc uniquement sur l'apprentissage de l'opérateur linéaire Γ .

4.2.2 Généralisation de l'ajustement du pas d'apprentissage

Dans 3.3.2, nous ajustons manuellement le pas d'apprentissage η afin de faciliter la convergence du modèle. Cependant, cette approche reste assez spécifique au pays étudié. Des méthodes plus générales existent, comme le **Cosine Annealing** [22], qui ajuste le pas de descente au fil des *epochs*. Introduit relativement récemment, le **Cosine Annealing** propose une variation périodique du pas de descente en fonction du nombre d'*epochs* :

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{T_{\text{cur}}}{T_{\text{max}}} \pi \right) \right)$$

Cette approche offre une alternative à l'adaptation manuelle proposée en Section 3.3.2 lors de l'apprentissage sur la base de données, et permet ainsi de traiter simultanément plusieurs pays. Cette modification ne change pas les résultats établis dans la Section 3, voir Annexe D.

4.3 Apprentissage sur une base de données

4.3.1 Ajustement à plusieurs séries temporelles

Dans un premier temps, nous cherchons à estimer simultanément des nombres de reproduction linéaires par morceaux sur plusieurs séries temporelles de nombres de cas. L'entraînement sera

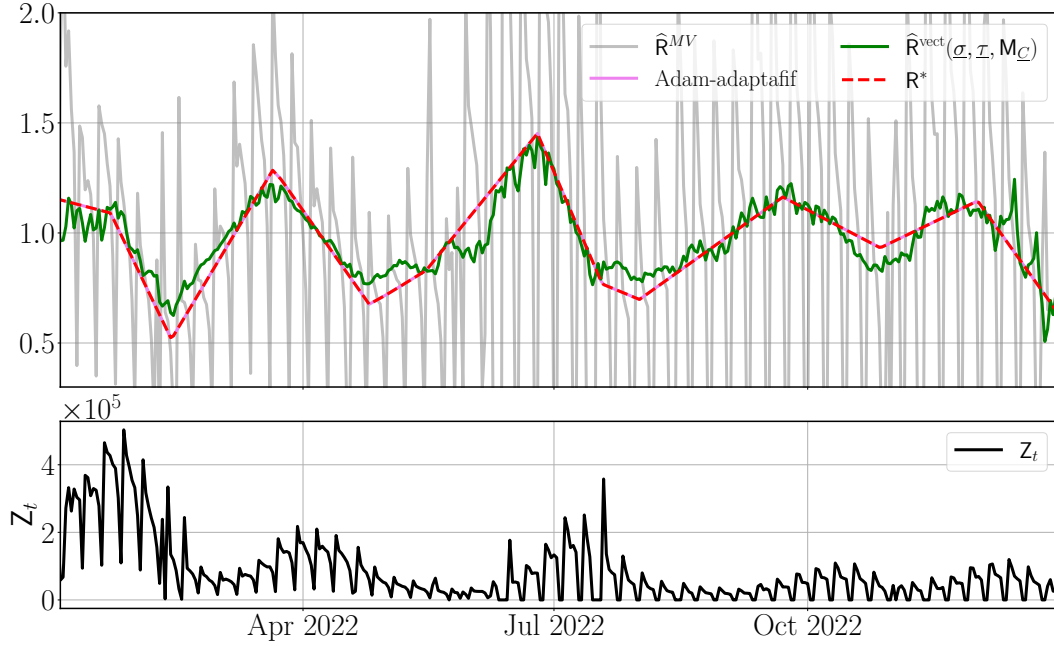


FIGURE 21 – Apprentissage sur une série temporelle avec 10 couches pour le réseau déroulé.

Première ligne : *En rose* : estimateur avec apprentissage de Γ . *En vert* : estimateur avec apprentissage de $\underline{\sigma}, \underline{\tau}, \underline{M}_C$. *En rouge* : oracle. *En gris* : estimateur du maximum de vraisemblance.

Deuxième ligne : décompte journalier des nouveaux cas Z_t .

réalisé sur un ensemble de 16 pays (donc $P = 16$ dans la perte (E3)), qui constituent notre base de données d'entraînement ayant été créée comme décrit dans la Section 4.1. Nous créons le réseau déroulé issu de l'algorithme de Chambolle-Pock comme décrit en Section 3, avec $k_{\text{layer}} = 10$, puis nous résolvons le problème (E3) avec une descente de gradient **Adam**. Etant donné la complexité de l'entraînement sur une base de données, nous avons porté le nombre d'*epochs* de 5 000 à 10^5 (ce qui renforce par ailleurs notre volonté de réduire le nombre de couches du réseau).

Sur la Figure 23a, on observe l'évolution de la perte au cours des *epochs* lors de l'apprentissage de l'opérateur linéaire représenté par la matrice pleine Γ commune à toutes les couches du réseau. A l'aide de l'optimiseur **Adam**, en orange, ainsi que celle obtenue avec **Adam** couplé à une stratégie d'adaptation du taux d'apprentissage via **Cosine Annealing**, en bleu. Sans stratégie d'adaptation, le comportement observé rejoint celui de la Figure 19. En revanche, bien que **Adam** avec **Cosine Annealing** parvienne à échapper à ce problème, la convergence reste très lente. Avec notre budget fixé à 10^5 *epochs*, le modèle entraîné sans adaptation du taux d'apprentissage s'adapte mieux aux données d'apprentissage. Poursuivre l'entraînement davantage pour observer l'effet complet de la stratégie **Cosine Annealing** semble difficilement envisageable dans ce contexte : l'entraînement sur 10^5 *epochs* nécessite déjà environ 10 heures de calcul. La Figure 22 présente l'estimateur obtenu pour un pays de la base de données dans les deux configurations : en orange sans adaptation du pas de descente, et en bleu avec la stratégie **Cosine Annealing**. Nous pouvons constater dans les deux cas une structure bien linéaire par morceaux, très proche de l'oracle visé.

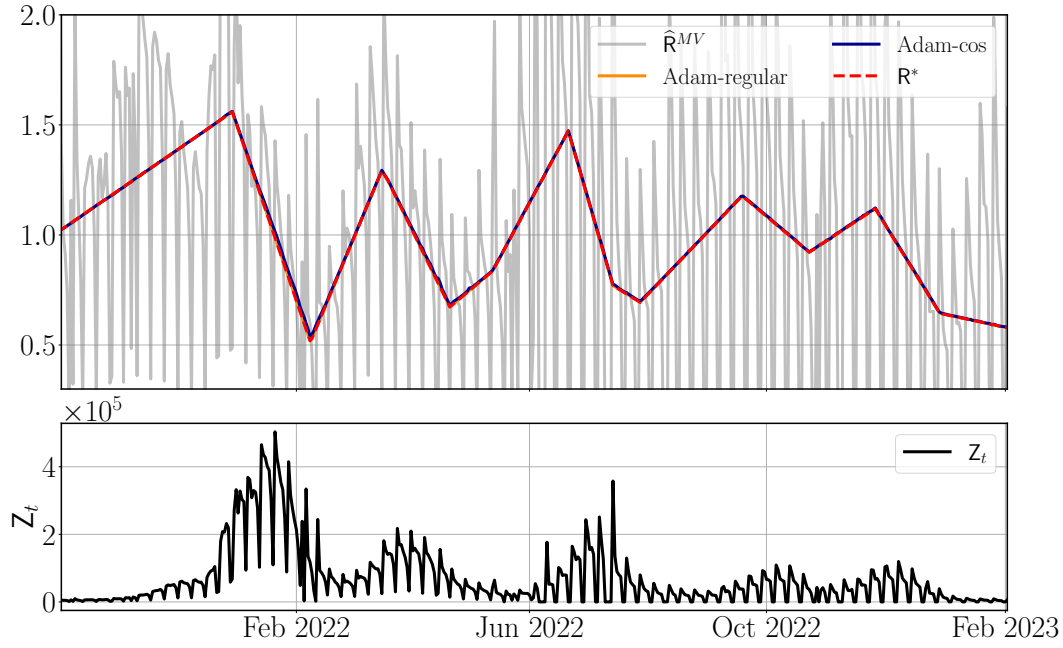


FIGURE 22 – Première ligne : estimateurs du nombre de reproduction obtenus avec Adam. *En orange* : sans stratégie d’adaptation du pas de descente. *En bleu* : avec stratégie du pas de descente. *En rouge* : oracle. Deuxième ligne : décompte journalier des nouveaux cas Z_t .

4.3.2 Etude de la capacité de généralisation

Nous avons désormais montré que le modèle peut s’ajuster simultanément à plusieurs oracles de nombre de reproduction ; nous allons à présent étudier sa capacité de généralisation. Lors de l’apprentissage des paramètres, la perte définie dans le problème (E3) est évaluée à la fois sur l’ensemble d’entraînement et sur un jeu de validation, constitué de données jamais vues par le modèle durant l’entraînement. A chaque *epoch*, nous comparons les pertes sur ces deux ensembles afin d’évaluer conjointement la performance d’approximation sur la base d’entraînement et la capacité de généralisation sur la base de validation. L’entraînement est réalisé sur les 16 pays sélectionnés dans la sous-section précédente, tandis que la validation s’effectue sur 4 autres pays distincts (donc $P = 4$ dans la perte (E3)). La répartition entre les deux ensembles est figée : chaque pays appartient exclusivement soit à l’ensemble d’entraînement, soit à celui de validation, sans changement au cours du processus.

La Figure 23b montre l’évolution de la perte sur le jeu de validation. On retrouve respectivement en bleu et orange les courbes correspondant aux apprentissages avec et sans stratégie d’adaptation du pas. Le modèle sans adaptation, en orange, présente une dynamique beaucoup plus erratique, rendant l’analyse difficile. En revanche, la courbe en bleu, correspondant à l’optimisation avec **Cosine Annealing**, révèle un comportement plus interprétable. Au début de l’entraînement, le réseau ajuste ses poids pour s’adapter aux données d’apprentissage, construisant ainsi progressivement un estimateur linéaire par morceaux. Ce processus s’accompagne d’une amélioration des performances sur le jeu de validation. Toutefois, après environ 50 000 itérations, on constate en Figure 23b, courbe en bleu, une augmentation de la perte de validation, signe d’un sur-apprentissage. Le modèle commence alors à se spécialiser excessivement sur les données d’entraînement, au détriment de sa capacité à généraliser. On peut donc supposer que continuer l’entraînement n’améliorera pas

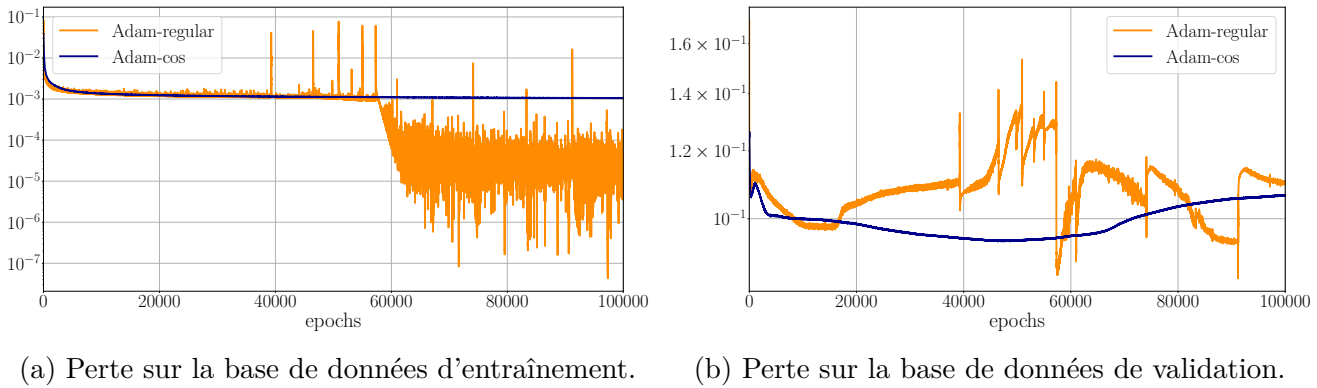


FIGURE 23 – Evolution de la perte (E3) au fur et à mesure des *epochs*. *En orange* : avec Adam. *En bleu* : avec Adam et une stratégie d'adaptation de pas de descente Cosine Annealing.

la capacité de généralisation du modèle appris. Cela pourrait être dû au faible nombre de données d'entraînement ou à l'insuffisance de poids apprenables.

5 Conclusions et perspectives

Le suivi d'une épidémie repose sur l'étude d'indicateurs épidémiologiques essentiels, tels que le nombre de reproduction quantifiant l'intensité de la propagation du virus. Dans le cas de la pandémie de COVID-19, le bruit administratif présent dans les données complique fortement cette analyse. Il demande l'utilisation d'estimateurs avancés tels que les estimateurs variationnels permettant d'imposer une certaine régularité temporelle à l'estimée, calculés au moyen d'un algorithme d'optimisation. Il impose également des choix paramétriques cruciaux, comme le choix du type précis de régularité attendue, tel que la linéarité par morceaux obtenue en pénalisant la dérivée seconde de l'estimée, et du niveau de régularisation. Une alternative consiste à apprendre automatiquement ces paramètres grâce au dépliement sous forme de réseau de neurones de l'algorithme sous-jacent. Nos premières expérimentations, menées sur une unique série temporelle, ont montré qu'il n'est pas aisé de s'approcher des oracles et qu'un réseau déroulé doit disposer d'un nombre suffisant de paramètres pour approcher des oracles suffisamment régularisés. Nous avons néanmoins mis en évidence des jeux de poids apprenables qui permettent d'atteindre une bonne approximation après apprentissage sur une série temporelle. Afin de tester la capacité de généralisation du réseau déroulé, nous avons créé une base de données en sélectionnant finement des comptes réels d'infection à la COVID-19 et en pré-traitant ces données pour éviter les écueils numériques. Lorsque l'entraînement est étendu à cette base de données composées de plusieurs séries temporelles, il apparaît que l'application du dépliement d'algorithmes aux séries temporelles reste délicate : il faut un nombre important de paramètres pour atteindre de bonnes capacités d'approximation sur toute la base de données. Les difficultés sont particulièrement marquées lorsque le réseau est testé sur de nouvelles données sans ré-entraînement préalable : les capacités de généralisation sont pour le moment limitées. En perspective, il serait donc nécessaire d'approfondir l'étude des capacités de généralisation des réseaux déroulés appliqués aux séries temporelles. Plus précisément, le choix du pas d'apprentissage ainsi que celui des paramètres du réseau déroulé peuvent être remis en question : peut-on notamment accepter de réduire légèrement la capacité d'approximation afin de renforcer la capacité de généralisation ? Une autre piste serait également de s'inspirer de l'architecture des réseaux de convolution. Au lieu de considérer un opérateur linéaire encodant

un unique filtre de taille 25 tel que M_C , nous pourrions apprendre plusieurs filtres par couche afin d'augmenter le nombre de poids par couche et donc la complexité du réseau. Il s'agit alors de trouver un compromis entre interprétabilité et complexité : choisissons-nous de rester dans le cadre interprétable de l'algorithme déroulé n'apprenant que λ , ou de nous tourner vers des réseaux convolutifs plus généraux, mais moins transparents, à l'effet « boîte noire ».

Références

- [1] J. WALLINGA et P. TEUNIS, « Different Epidemic Curves for Severe Acute Respiratory Syndrome Reveal Similar Impacts of Control Measures, » *Am. J. of Epidemiol.*, t. 160, n° 6, p. 509-516, 2004.
- [2] A. CORI, N. M. FERGUSON, C. FRASER et S. CAUCHEMEZ, « A new framework and software to estimate time-varying reproduction numbers during epidemics, » *American journal of epidemiology*, t. 178, n° 9, p. 1505-1512, 2013.
- [3] R. N. THOMPSON et al., « Improved inference of time-varying reproduction numbers during infectious disease outbreaks, » *Epidemics*, t. 29, p. 100-105, 2019.
- [4] P. ABRY et al., « Spatial and temporal regularization to estimate COVID-19 reproduction number $R(t)$: Promoting piecewise smoothness via convex optimization, » *Plos one*, t. 15, n° 8, 2020.
- [5] B. PASCAL, P. ABRY, N. PUSTELNIK, S. ROUX, R. GRIBONVAL et P. FLANDRIN, « Nonsmooth convex optimization to estimate the Covid-19 reproduction number space-time evolution with robustness against low quality data, » *IEEE Transactions on Signal Processing*, t. 70, p. 2859-2868, 2022.
- [6] V. MONGA, Y. LI et Y. C. ELDAR, « Algorithm unrolling : Interpretable, efficient deep learning for signal and image processing, » *IEEE Signal Processing Magazine*, t. 38, n° 2, p. 18-44, 2021.
- [7] F. RICCARDO et al., « Epidemiological characteristics of COVID-19 cases and estimates of the reproductive numbers 1 month into the epidemic, Italy, 28 January to 31 March 2020, » *Eurosurveillance*, t. 25, n° 49, p. 2000790, 2020.
- [8] D. CEREDA et al., « The early phase of the COVID-19 epidemic in Lombardy, Italy, » *Epidemics*, t. 37, p. 100528, 2021.
- [9] N. PUSTELNIK, C. CHAUX et J.-C. PESQUET, « Hybrid Regularization for Data Restoration in the Presence of Poisson Noise, » in *Proceedings of the 17th European Signal Processing Conference (EUSIPCO)*, 2009.
- [10] M. EL GHECHE, J.-C. PESQUET et J. FARAH, « A Proximal Approach for Optimization Problems Involving Kullback Divergences, » in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Vancouver, Canada, 2013.
- [11] A. CHAMBOLLE et T. POCK, « A first-order primal-dual algorithm for convex problems with applications to imaging, » *Journal of mathematical imaging and vision*, t. 40, n° 1, p. 120-145, 2011.
- [12] K. D. B. J. ADAM et al., « A method for stochastic optimization, » *arXiv preprint arXiv:1412.6980*, t. 1412, n° 6, 2014.

- [13] H. ROBBINS et S. MONRO, « A stochastic approximation method, » *The annals of mathematical statistics*, p. 400-407, 1951.
- [14] K. GREGOR et Y. LECUN, « Learning fast approximations of sparse coding, » in *Proceedings of the 27th international conference on international conference on machine learning*, Haifa, Israel, 2010, p. 399-406.
- [15] R. TIBSHIRANI, « Regression Shrinkage and Selection Via the Lasso, » *Journal of the Royal Statistical Society : Series B (Methodological)*, t. 58, n° 1, p. 267-288, déc. 2018.
- [16] I. DAUBECHIES, M. DEFRISE et C. DE MOL, « An iterative thresholding algorithm for linear inverse problems with a sparsity constraint, » *Communications on Pure and Applied Mathematics : A Journal Issued by the Courant Institute of Mathematical Sciences*, t. 57, n° 11, p. 1413-1457, 2004.
- [17] C. CHAUX, A. SINGH, E. SOUBIES et V. Y. TAN, « Algorithm unrolling for solving inverse problems in signal and image processing, » in *AI4X 2025 International Conference*, Singa-pour, 2025.
- [18] P. L. COMBETTES et J.-C. PESQUET, « Deep neural network structures solving variational inequalities, » *Set-Valued and Variational Analysis*, t. 28, n° 3, p. 491-518, 2020.
- [19] Y. HUANG, E. CHOUZENOUX et J.-C. PESQUET, « Unrolled variational Bayesian algorithm for image blind deconvolution, » *IEEE Transactions on Image Processing*, t. 32, p. 430-445, 2022.
- [20] H. T. V. LE, A. REPETTI et N. PUSTELNIK, « Unfolded proximal neural networks for robust image Gaussian denoising, » *IEEE Transactions on Image Processing*, t. 33, p. 4475-4487, 2024.
- [21] Y. ATCHADE, X. LIU et Q. ZHU, « A statistical perspective on algorithm unrolling models for inverse problems, » *arXiv preprint arXiv :2311.06395*, 2023.
- [22] I. LOSHCHILOV et F. HUTTER, « Sgdr : Stochastic gradient descent with warm restarts, » *arXiv preprint arXiv :1608.03983*, 2016.
- [23] P. L. COMBETTES et V. R. WAJS, « Signal recovery by proximal forward-backward splitting, » *Multiscale modeling & simulation*, t. 4, n° 4, p. 1168-1200, 2005.

A Notations

La Table 2 résume les notations et symboles principalement utilisés dans le document

Terme	Description
Z_t	Nombre de contamination au jour t
R_t	Nombre de reproduction au jour t
Φ_t^Z	Infectiosité globale au jour t
$\mathbf{Z}$	Vecteur des nombres de contamination ($Z_1, \dots, Z_T$)
$\mathbf{R}$	Vecteur des nombres de reproduction ($R_1, \dots, R_T$)
Φ^Z	Vecteur d'infectiosité globale
ϕ	Fonction d'intervalle de série
D_2	Opérateur de dérivée seconde discret
λ	Paramètre de régularisation
σ, τ	Paramètres algorithmiques
prox_f	Opérateur proximal de la fonction f
f^*	Conjuguée convexe de la fonction f
d_{KL}	Divergence de Kullback-Leibler pour une ob- servation Z_t
D_{KL}	Divergence de Kullback-Leibler pour le vec- teur $\mathbf{Z}$
$\mathbf{I}_n$	Matrice identité de taille n
$\mathbf{0}_{n,m}$	Matrice de zéros avec n lignes et m colonnes
Id	Fonction identité $f(x) = x$
k_{layer}	Nombre de couche du réseau
$\underline{x}$	Vecteur de taille k_{layer}
$\hat{\mathbf{R}}^{MV}$	Estimateur donné par le maximum de vrai- semblance
$\hat{\mathbf{R}}^{\text{Cori}}$	Estimateur donné par la moyenne a poste- riori avec a priori Gamma
$\hat{\mathbf{R}}^{\text{CP}}(\lambda, \sigma, \tau)$	Estimateur donné par l'algorithme de Chambolle-Pock 2
$\mathcal{NN}_{\Theta}(\mathbf{Z})$	Estimateur obtenu avec le réseau de neurones déroulé paramétré par Θ
$\mathbf{R}^*$	Oracle utilisé pour lors de l'entraînement du réseau de neurones déroulé
$x \odot y$	Produit de Hadamard entre le vecteur x et le vecteur y

TABLE 2 – Notations et symboles

B Descente de gradient et Adam

La descente de gradient est une méthode d'optimisation itérative utilisée pour minimiser une fonction objectif différentiable. Elle est couramment employée en apprentissage statistique pour ajuster les paramètres d'un réseau de neurones afin de minimiser l'erreur entre les prédictions du modèle et les valeurs réelles. Il est également possible d'utiliser cette technique lorsque les fonctions d'activations du réseau sont non-différentiables en utilisant la sous-différentielle (voir Définition 2).

B.1 Principe général

Etant donné une fonction de coût $\mathcal{L}(\theta)$, où θ représente les paramètres du modèle, l'objectif est de trouver les valeurs de θ qui minimisent cette fonction. La descente de gradient procède en mettant à jour les paramètres dans la direction opposée au gradient de la fonction de coût par rapport à ces paramètres. Le gradient indique en effet la direction de la plus forte augmentation de la fonction ; en prenant l'opposé, on se dirige vers un minimum

$$\theta^{[k+1]} := \theta^{[k]} - \alpha \nabla_{\theta} \mathcal{L}(\theta^{[k]})$$

où :

- θ : vecteur des paramètres du modèle,
- α : taux d'apprentissage (*learning rate*),
- $\nabla_{\theta} \mathcal{L}(\theta^{[k]})$: gradient de la fonction de coût évalué en $\theta^{[k]}$.

B.2 Algorithme de base

Algorithme 4 Méthode de descente de gradient

Initialisation $\theta \in \text{dom } \mathcal{L}$

Tant que Un critère d'arrêt n'est pas atteint **faire** :

1. *Calcul.* On calcule $\nabla_{\theta} \mathcal{L}(\theta^{[k]})$
2. *Mise à jour.* $\theta^{[k+1]} := \theta^{[k]} - \alpha \nabla_{\theta} \mathcal{L}(\theta^{[k]})$

Fin

Le comportement typique de l'Algorithme 4 est représenté sur la Figure 24.

B.3 Algorithme Adam

Afin de palier la lenteur de l'algorithme de descente de gradient et son possible piégeage dans des minima locaux, un algorithme alternatif appelé **Adam** a été proposé [12]. Cet algorithme reprend le principe de descente de gradient, en ajoutant notamment une notion de momentum. Si les gradients vont dans la même direction plusieurs fois, **Adam** les « accumule », ce qui accélère l'optimisation dans les bonnes directions.

L'algorithme est décrit par l'Algorithme 5, où α (learning rate), β_1 , β_2 (taux de décroissance exponentielle pour les estimations du moment) et ϵ (pour éviter les divisions par 0) sont des hyperparamètres à régler, typiquement :

$$\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999 \text{ et } \epsilon = 1e^{-8}$$

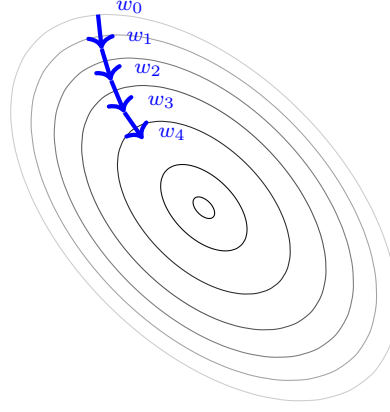


FIGURE 24 – Descente de gradient classique. *En noir* : ligne de niveau. *En bleu* : itération de l'algorithme. *Figure copié de github.com/davidstutz/latex-resources/.*

Algorithme 5 Adaptive Moment Estimation (Adam)

Initialisation $\theta \in \text{dom } \mathcal{L}$, $m_0 = 0$, $v_0 = 0$, $t = 0$

Tant que Un critère d'arrêt n'est pas atteint **faire** :

1. $t = t + 1$
2. Calcul du gradient $g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$
3. Mise à jour du premier moment : $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$
4. Mise à jour du second moment : $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
5. Correction du biais : $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$, $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$
6. Mise à jour des paramètres : $\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$

Fin

Remarque 2 $\beta_{1,2}^t$ représente $\beta_{1,2}$ à la puissance t , et toutes les opérations sur des vecteurs sont faites composante par composante. Notamment g_t^2 représente le produit de Hadamard $g_t \odot g_t$

On peut observer la différence de comportement entre une descente classique et Adam sur la Figure 25



(a) Descente de gradient classique. *En noir* : ligne de niveau. *En bleu* : itération de l'algorithme.

(b) Descente de gradient Adam. *En noir* : ligne de niveau. *En bleu* : itération de l'algorithme.

FIGURE 25 – Comparaison des methodes Classique et Adam.

C Concepts et outils d'optimisation convexe non lisse

Le but de cette section est de donner les grands principes de la résolution de problèmes de minimisation d'une fonctionnelle convexe non lisse de type

$$\hat{x} \in \underset{x \in \mathcal{H}}{\operatorname{Argmin}} f(x) \quad (13)$$

Définition 1 On dit qu'une fonction f est une fonction propre lorsque son domaine est non vide :

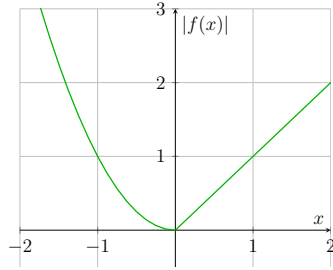
$$\operatorname{dom}(f) \neq \emptyset$$

où $\operatorname{dom}(f) = \{x \mid f(x) \text{ est définie}\}$

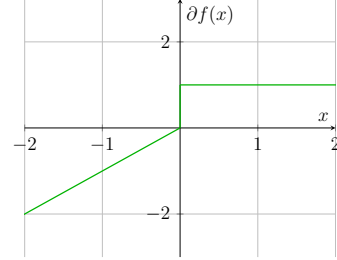
Définition 2 Soit $f : \mathcal{H} \rightarrow]-\infty, +\infty]$ une fonction propre. La sous-différentielle de f , notée ∂f , est définie par

$$\begin{aligned} \partial f : \mathcal{H} &\rightarrow 2^{\mathcal{H}} \\ x &\mapsto \{u \in \mathcal{H} : \forall y \in \mathcal{H} \langle y - x, u \rangle + f(x) \geq f(y)\} \end{aligned}$$

Cela permet notamment de « multi-évaluer » le gradient d'une fonction qui aurait eu un point angulaire :



(a) Fonction $f(x) = x^2 \mathbf{1}_{\mathbb{R}^-} + x \mathbf{1}_{\mathbb{R}^+}$.



(b) Opérateur sous-différentielle

FIGURE 26 – Fonction avec un point angulaire et sa sous-différentielle.

Théorème 1 Règle de Fermat : Soit $f : \mathcal{H} \rightarrow]-\infty, +\infty]$ une fonction propre, alors

$$0 \in \partial f \iff x \in \operatorname{Argmin} f$$

Définition 3 Soit $f : \mathcal{H} \rightarrow]-\infty, +\infty]$. L'épigraphe de f est donné par

$$\operatorname{epi}(f) = \{(x, \xi) \in \operatorname{dom}(f) \times \mathbb{R} : f(x) \leq \xi\}$$

Définition 4 Soit $f : \mathcal{H} \rightarrow]-\infty, +\infty]$. On dit que f est une fonction semi-continue inférieure lorsque $\operatorname{epi}(f)$ est un fermé de $\mathcal{H}$ au sens topologique⁸.

8. Ceci est plutôt une équivalence, il existe une autre définition plus complexe de la semi-continuité inférieure, mais nous n'en avons pas besoin ici.

Définition 5 On note $\Gamma_0(\mathcal{H})$ l'ensemble des fonctions propres, convexes, semi-continues inférieurement à valeurs dans $]-\infty, +\infty]$ définies sur l'espace de Hilbert $\mathcal{H}$.

Définition 6 Soient $\mathcal{H}$ un espace de Hilbert et $f \in \Gamma_0(\mathcal{H})$. L'opérateur proximal de f est défini par

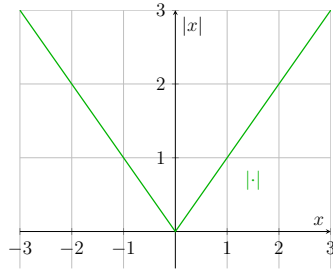
$$\begin{aligned} \text{prox}_{\gamma f} : \mathcal{H} &\longrightarrow \mathcal{H} \\ x &\longmapsto \underset{y \in \mathcal{H}}{\text{argmin}} f(y) + \frac{1}{2\gamma} \|x - y\|_2^2 \end{aligned}$$

Dans notre cas, on applique par exemple une pénalisation ℓ_1 de la forme $\lambda \|x\|_1 = \lambda \sum_{i=1}^n |x_i|$.

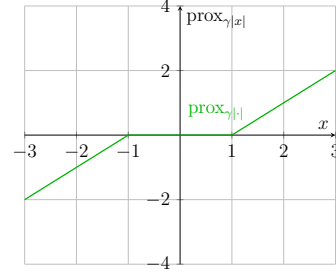
Il peut alors être intéressant de regarder l'expression de l'opérateur proximal de la fonction $|\cdot|$. L'opérateur proximal de la valeur absolue est donnée par

$$\text{prox}_{\gamma|\cdot|}(x) = \text{sign}(x) \max(|x| - \gamma, 0)$$

On peut retrouver respectivement la représentation graphique de la fonction valeur absolue ainsi que son opérateur proximal sur les Figure 27a et 27b



(a) Fonction $|\cdot|$.



(b) Fonction $\text{prox}_{\gamma|\cdot|}$.

FIGURE 27 – Fonction valeur absolue et son opérateur proximal.

Proposition 2 Soient $\mathcal{H}$ un espace de Hilbert et $f \in \Gamma_0(\mathcal{H})$. Alors pour $\rho \in \mathbb{R}^*$, l'opérateur proximal de $g(x) = f(\rho x)$ est

$$\frac{1}{\rho} \text{prox}_{\rho^2 f}(\rho x)$$

Exemple 1 Opérateur proximal de la divergence de Kullback-Leibler pour une variable aléatoire X suivant une loi de Poisson de paramètre α :

La divergence de Kullback-Leibler s'écrit :

$$d_{KL}(\alpha, Z_t) = Z_t \ln\left(\frac{Z_t}{\alpha}\right) - Z_t + \alpha$$

On cherche donc à calculer

$$\begin{aligned} \text{prox}_{\gamma \ell}(x) &= \underset{y \in \mathbb{R}}{\text{argmin}} \frac{1}{2\gamma} |x - y|^2 + \ell(y, Z_t) \\ \iff 1 - \frac{Z_t}{y} + \frac{1}{\gamma}(y - x) &= 0 \\ \iff y^2 + (\gamma - x)y - \gamma Z_t &= 0 \\ \iff y &= \frac{x - \gamma + \sqrt{\Delta}}{2} \end{aligned}$$

où $\Delta = (\gamma - x)^2 + 4\gamma Z_t$. Pour une loi de Poisson de paramètre $\alpha = R_t \Phi_t^Z$, en vertu de la Proposition 2 :

$$\text{prox}_{\gamma d_{\text{KL}}}(R_t) = \frac{R_t - \gamma \Phi_t^Z + \sqrt{(R_t - \gamma \Phi_t^Z)^2 + 4\gamma Z_t}}{2} \quad (14)$$

Un opérateur proximal similaire a été calculé dans l'exemple 2.18 de [23].

Exemple 2 Opérateur proximal de l'oposée de la log-vraisemblance, utilisé comme attache aux données, d'une variable aléatoire suivant une loi normale de moyenne α et de variance α , pour $\alpha \gg 1$:

La densité d'une loi normale de moyenne α et de variance α est

$$f(x) = \frac{1}{\sqrt{2\pi\alpha}} \exp\left(-\frac{(x - \alpha)^2}{2\alpha}\right)$$

La log-vraisemblance pour une observation x est donc

$$\ell(\alpha, x) = -\frac{1}{2} \log(2\pi\alpha) - \frac{(x - \alpha)^2}{2\alpha}$$

Maximiser la log-vraisemblance ℓ revient donc à minimiser $g(\alpha) = \frac{(x - \alpha)^2}{2\alpha}$. Lorsque $Z_t \simeq \alpha$, on cherche donc à calculer

$$\begin{aligned} \text{prox}_{\tau g}(x) &= \underset{y \in \mathbb{R}}{\text{argmin}} \frac{1}{2\tau} |x - y|_2^2 + \frac{1}{2Z_t} (y - Z_t)^2 \\ \iff \frac{1}{Z_t} (Z_t - y) + \frac{1}{\tau} (x - y) &= 0 \\ \iff y &= \frac{1 + x/\tau}{1/Z_t + 1/\tau} \end{aligned}$$

on obtient finalement :

$$\text{prox}_{\tau g}(R_t) = \frac{\tau + R_t}{1 + \tau/Z_t} \quad (15)$$

Proposition 3 Soit $\mathcal{H}$ un espace de Hilbert et $f \in \Gamma_0(\mathcal{H})$, alors

$$\forall x \in \mathcal{H}, \quad p = \text{prox}_f(x) \iff x - p \in \partial f(p)$$

Pour résoudre le problème (13) par descente de gradient avec un schéma implicite :

$$x_{n+1} = x_n - \gamma \nabla f(x_{n+1})$$

On prend $u_n \in \partial f(x_{n+1})$ et alors le schéma est équivalent à

$$\begin{aligned} x_{n+1} + \gamma u_n &= x_n \\ \iff (Id + \gamma \partial f)(x_{n+1}) &= x_n \\ \iff x_{n+1} &= \text{prox}_{\gamma f}(x_n) \end{aligned}$$

Proposition 4 Identité de Moreau : Soit $\varphi : X \longrightarrow \mathbb{R} \cup \{\infty\}$ convexe, propre et semi-continue inférieurement et φ^* sa conjuguée convexe, alors pour tout réel $\tau > 0$, pour tout élément $x \in X$:

$$x = \tau \text{prox}_{\varphi/\tau}\left(\frac{x}{\tau}\right) + \text{prox}_{\tau\varphi^*}(x). \quad (16)$$

D Stratégie d'adaptation du pas d'apprentissage

Cette annexe confirme que l'adoption d'une stratégie d'adaptation du pas d'apprentissage via **Cosine Annealing** ne modifie pas les conclusions établies en Section 4.2.1. Lorsqu'on cherche à estimer une série temporelle à partir de l'apprentissage de Γ , la Figure 28 montre qu'avec cette méthode et un réseau de $k_{\text{layer}} = 10$ couches, l'oracle $\mathbf{R}^*$ reste correctement approché. Par ailleurs, la Figure 29 illustre que, tout comme avec un ajustement du pas de descente basé sur la perte, **Cosine Annealing** permet également d'atteindre une erreur nulle après apprentissage. Nous utiliserons **Cosine Annealing** pour $\eta_{\min} = 5 \times 10^{-5}$, $\eta_{\max} = 1 \times 10^{-4}$ et $T_{\max} = 100$

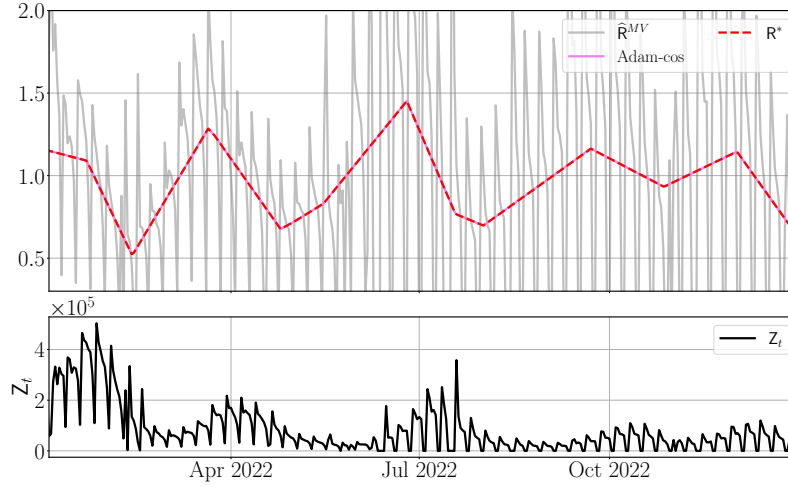


FIGURE 28 – Apprentissage de l'opérateur linéaire Γ sur une série temporelle avec 10 couches pour le réseau déroulé, et l'optimiseur Adam couplé à **Cosine Annealing**.

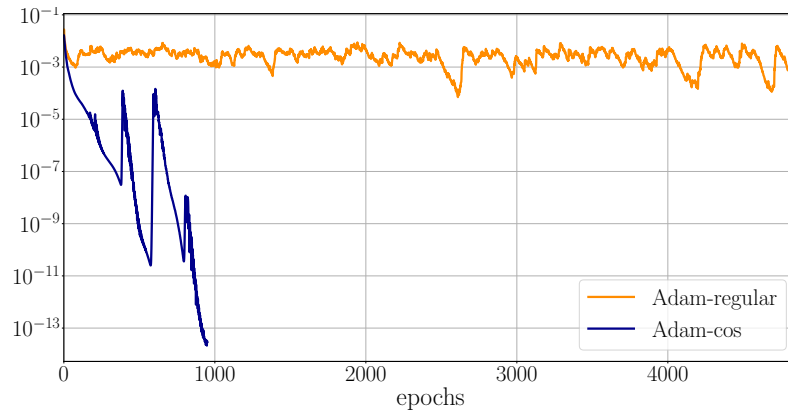


FIGURE 29 – Evolution de la perte (E) au fur et à mesure des *epochs*. *En orange* : avec Adam. *En bleu* : avec Adam et une stratégie d'adaptation de pas de descente **Cosine Annealing**.

E Illustration des données

